CHURCH OF MALWARE / FIELD MANUAL



RE for beginners with SPECTRA

Follow the bytes. Test an idea. Keep the evidence.

Start from zero: binary basics, Windows, Linux, ARM, .NET, Android, saved investigations three guided CTF campaigns and a bridge to later cases.

EDITION 3 / 11 NOVEMBER 2026
SPECTRA 0.18 / BENIGN PRACTICE FILES INCLUDED
Training Ground edition / campaigns 001-008

Contents

1. You belong at this workbench 4

2. What RE actually does 6

3. Bytes, hex, text, and numbers 8

4. A tiny computer inside your head 10

5. What is inside an executable? 12

6. Your first lap around SPECTRA 14

7. One byte, three addresses: learn the map 16

8. Case file: a Windows PE from bytes to meaning 18

9. Case file: the Linux marker that does not call home 20

10. Case file: ARM, two numbers, eight bytes 22

11. Read reconstructed code without getting fooled 24

12. Debugging means controlling a question 26

13. Case file: watch the Windows model work 28

14. Case file: Linux reads, writes, and exits 30

15. Case file: catch a string being built 32

16. Rewind a small experiment without losing your place 34

17. Case file: a .NET client you can read 36

18. Case file: an Android package, from manifest to method bytes 38

19. Encoded, compressed, packed, encrypted: choose the right question 41

20. Case file: a firmware package with another program inside 43

21. Case file: one changed instruction, two program versions 45

22. Your first real malware investigation 47

23. Keep your work so tomorrow starts ahead 49

24. Save a run, inspect a trace, and know what survived 52

25. Enter the CTFs: solve a question, not a guessing game 54

26. The Dead Channel: your first five evidence trails 56

27. Glass Relay: maps, references and bounded records 62

28. Cold Storage: investigate without inventing a malware story 68

29. From first cases to connected investigations 76

30. From packed bytes to interpreted behavior 78

31. Every section: the desk reference 80

32. When the tool disagrees with you 84

33. Finish an investigation you can defend 86

34. The vocabulary, without the gatekeeping 87

35. Sources and laboratory notes 91

1. You belong at this workbench

You do not need an expensive license, a shelf of computers, or a job title to start reverse engineering. You need a question, something to inspect, and a way to keep track of what you learn. This book gives you all three. We will begin with tiny harmless programs and work toward a defensible investigation of an unfamiliar file.

Reverse engineering (RE) means working backward from an artifact to understand its design and behavior. Our artifacts are mostly compiled programs: files whose original source code may be missing. We inspect bytes, recover instructions, follow relationships, reconstruct useful explanations, and test small hypotheses. The goal is understanding you can show someone else how to reproduce.

That can mean figuring out how malware stores its configuration. It can also mean understanding an old file format, checking a vendor's patch, documenting a device's firmware, or finding which part of your own desktop client builds a request. A suspicious file is one use case. Curiosity is a much bigger one.

How this book works

Read chapters 1-6 in order. They explain the vocabulary used everywhere else. Then complete the Windows, Linux, and ARM investigations. Later chapters introduce controlled execution, managed code, decoding, firmware, and comparisons. Each case has a specific question, steps, expected observations, a notebook entry, and a checkpoint. Answers are included so you can learn without waiting for someone to grade you.

Bold text names a SPECTRA control. Monospaced text is an exact value, filename, instruction, or example. Addresses beginning with `0x` are hexadecimal. A value labeled **example** is teaching data, not a claim about an unknown malware sample. Save your current investigation before loading any demo: demos replace the active file.

Your setup

1. Open SPECTRA in a current browser. The book describes release 0.18, checked against its source and fixtures in November 2026. Future labels or limits may change.
2. Keep this manual beside it, or switch between tabs. On a phone, use the menu to select a section and scroll to the relevant controls. A wider view helps with assembly, but a keyboard is not required for the early lessons.
3. Go to **Training ground**. Keep **Explanations on** while learning. Use the built-in exercise buttons when a chapter names one.
4. Save useful work in **Notebook** and download a project before a long break. Mobile browsers can discard background tabs; browser storage is convenient, not a permanent backup.

The companion practice folder contains benign files for the additional lessons. You can use the built-in demos without downloading it. The book's HTML and PDF can be read offline; first use of SPECTRA's analysis engines needs their assets, unless you already have the complete offline release. Downloading one HTML page is not the same as downloading all engines.

A realistic promise

SPECTRA brings useful static analysis and bounded emulation into the browser. It is not every operating system or device in a tab. A supported CPU does not imply every instruction, executable, library, or system call will work. Learning to recognize the boundary is part of learning RE. When something stops, we will ask what assumption failed instead of treating the stop as your failure.

First checkpoint: Write one question in Notebook before opening a real sample. Good: “Which bytes become the configuration path?” Too broad: “Tell me everything about this malware.” A small question gives your investigation a finish line.

Choose your route

This edition follows SPECTRA 0.18 and the eight public CTF campaigns available on 11 November 2026. Start with the foundations and Windows/Linux/ARM cases, then try the Android package and the first three CTF campaigns. You can read a single case at a time; there is no prize for rushing past an unfamiliar word. Keep a small note of the next question before you close the tab.

Teaching is optional in SPECTRA. It defaults off; use the explanations button at the top to turn it on. The examples here explain the same evidence in more depth. Read these books alongside the workbench, or download the PDFs and practice files to keep learning offline.

2. What RE actually does

Imagine finding a small device with no manual. You can inspect its parts, follow its wires, and carefully test a button. Software gives us similar choices. **Static analysis** studies the file without running its instructions. **Dynamic analysis** observes an execution. SPECTRA's debugger executes supported instructions inside a model, which is a particular kind of experiment rather than a complete real computer.

Four views of the same thing

- **Bytes:** the stored data. Hex explorer shows these directly. They are the anchor when an interpretation gets confusing.
- **Assembly:** bytes decoded as instructions for a selected CPU. Disassembly answers what those instructions express, given the selected mode and mapping.
- **Reconstructed code:** a decompiler's higher-level explanation. It can expose conditions and loops, but names, types, and structure may be inferred.
- **Observed state:** registers, memory, output, and traces from one controlled run. These show what happened under the inputs and environment you supplied.

These views should support each other. Suppose reconstructed C says a loop XORs bytes with 0x5a. Assembly should contain the corresponding operations. A memory capture after the loop should show the predicted output. If they disagree, you have a useful question: wrong range, wrong type, wrong entry, wrong input, or an incomplete engine model?

Malware, without the movie version

Malware is software used to cause unauthorized harm: stealing information, disrupting systems, taking control, or enabling those actions. A loader may prepare another program. A downloader may fetch a payload. A backdoor may accept commands. A stealer may collect sensitive data. These roles can overlap, and a family label does not describe every file with that label.

Ordinary software also reads files, uses cryptography, talks to servers, and changes settings. Seeing `WriteFile`, a URL, or an encryption library is not enough to call a program malicious. Ask who initiates the behavior, what data it handles, which conditions lead there, and what authorization/context exists. RE supplies technical evidence; it does not turn one interesting string into intent.

The investigation loop

1. **Observe:** “This file contains the text /api/status at this offset.”
2. **Hypothesize:** “A routine may use it to construct an HTTP request.”
3. **Test:** follow references, inspect argument setup, identify the caller, and check whether that route reaches a networking operation.

4. **Revise:** perhaps it is a test fixture, documentation, or an unused error message.
5. **Record:** save the original evidence, your interpretation, and what remains untested.

A finding is stronger when another person can retrace it. “I found C2” is weak without context. “The routine at this VA copies this decoded hostname into this request object; actual network contact was not tested” is precise. C2 means command-and-control, but don't give a hostname that label until you have a reason.

Your first analyst habit

Keep three headings in every case: **Observed**, **Likely explanation**, and **Still unknown**. This is not academic fuss. It keeps your own guesses from becoming facts when you return tomorrow.

Checkpoint: You find the word “password.” What have you proven? **Answer:** Only that those bytes were extracted as text. You have not proved password theft, a valid credential, or even that the program uses the string. The next useful step is its code or data context.

3. Bytes, hex, text, and numbers

A file is a sequence of bytes. One byte contains eight bits; each bit is zero or one. A byte can represent a value from 0 through 255. We usually write bytes in hexadecimal because two hex digits describe one byte neatly. Hex digits run 0-9, then A-F for ten through fifteen.

`0x10` means sixteen, not ten. `0xff` means 255. `0x100` means 256. The prefix tells you the number base; it is not part of the stored bytes. A range with offset 0x20 and length 0x10 begins at byte 32 and contains 16 bytes, ending at offset 0x2f. The next byte is 0x30.

Read a hex row

```
offset   bytes                   possible text
0x0000   68 65 6c 6c 6f 00        hello.
```

The left column is a location, the middle is data, and the right is a convenient text interpretation. Here the five readable bytes spell `hello`. The zero byte may terminate a C-style string. A dot in a hex viewer usually means “not displayed as a printable character.” It does not always mean the underlying byte is zero.

One set of bytes, several interpretations

`41` can be decimal 65, the ASCII character A, part of an instruction, or one byte of a larger value. Context decides. A four-byte integer stored as `78 56 34 12` has value 0x12345678 in little-endian order. In big-endian order those same bytes have value 0x78563412. Endianness is the order of bytes inside a multi-byte value; it does not mean that a word is read backward letter by letter.

Unsigned numbers use all bits for nonnegative values. Signed two's-complement numbers interpret the top bit as part of a negative representation. The byte `ff` can mean unsigned 255 or signed -1. The instruction and declared type determine which interpretation matters. This is why a guessed signed comparison in decompiled code deserves a look at assembly.

Text is an encoding choice

ASCII covers a small character set. UTF-8 uses one or more bytes per character. UTF-16 uses two-byte code units and sometimes pairs of them. Plain English “Hi” can appear as `48 69` or, in UTF-16 little endian, `48 00 69 00`. Searching only one representation can miss the other. SPECTRA's wide-string extraction focuses on ASCII-range characters in UTF-16 patterns, so it is not a complete multilingual text decoder.

Encoding changes representation, such as Base64. **Compression** reduces redundancy, such as gzip. **Encryption** protects data using a key and an algorithm. All can make plain text disappear from the file. A successful Base64 decode does not mean you broke encryption.

Hashes and identity

A SHA-256 hash is a compact fingerprint computed from bytes. Renaming a file leaves its hash alone; changing its bytes normally changes the hash. Use it to identify exactly what you inspected. A hash is not a verdict, an author identity, or a guarantee of safety. When a download page and SPECTRA show different hashes, first check whether one describes the archive and the other describes the extracted member.

Mini-lab

1. Open **Decoder lab**. Choose **Hex → UTF-8 text**.
2. Enter `68 65 6c 6c 6f` and choose **Run transform**. Expect `hello`.
3. Choose Base64, enter `aGVsbG8=`, and run. Expect the same output.
4. Write down why the inputs differ even though their decoded text agrees.

Answer: They encode the same five bytes in different printable representations. Preserve the original representation and transform if you use this in an investigation. A copied output alone loses how you got there.

4. A tiny computer inside your head

You do not need to memorize a CPU manual. Start with a handful of ideas. A CPU fetches an instruction, operates on values, and chooses the next instruction. A **register** is a small named storage location inside that CPU. **Memory** is a larger addressed space. The program counter identifies the next instruction; stack and general registers support the work around it.

Instructions you will keep meeting

Job	What to look for	Useful question
Move, load, store	Copy a value or access memory	Is this a pointer, its contents, or a plain number?
Add, subtract, shift, XOR	Transform values	Which input contributes to this result?
Compare and conditional branch	Choose a path	Which condition takes the other path?
Call and return	Enter a routine and come back	What arguments went in, and what came out?

A **pointer** is a value used as an address. If R0 contains 0x2000, reading R0 gives you 0x2000. Reading memory at that address gives you the bytes stored there. Confusing the two causes many beginner debugger faults. If a routine expects a pointer to a string, giving it the number 3 is not giving it a three-character string.

Functions and the stack

A function is a reusable piece of work. It can accept arguments, use temporary values, call other functions, and return a result. The **stack** is memory commonly used for call bookkeeping and temporary storage. A **heap** is memory used for allocations with more flexible lifetimes. Global data lives in another part of the program's mapped memory.

A calling convention, part of an ABI, says where arguments and results belong. For our ARM addition lesson, inputs use R0 and R1 and the result uses R0. For an ordinary Windows x64 integer call, the first four argument positions use RCX, RDX, R8, and R9; a simple scalar result uses RAX. The caller also reserves stack space. This is why selecting the right ABI matters even when the CPU setting is correct. Microsoft's x64 calling convention and Arm's AAPCS32 describe the formal rules; the labs teach a small integer subset.

CPU is not operating system

Windows and Linux are operating systems. ARM, AArch64, x86, and x86-64 identify CPU instruction families/modes. A Windows ARM64 executable and a Linux ARM64 executable can target related instructions while expecting different loaders and APIs. A filename ending in .exe does not mean x86. An ELF file does not mean ARM. Read the header.

Thumb is an ARM instruction mode, not a different operating system. AArch64 is the 64-bit Arm instruction set. RISC-V and MIPS are other families. The same raw bytes decoded in the wrong mode can produce plausible-looking nonsense. You need evidence for CPU, byte order, address mapping, and environment before execution.

Loops and branches

A loop repeats instructions while a condition holds. A basic block is a run of instructions with no internal branch entry/exit except its boundaries. A control-flow graph connects blocks with possible paths. An arrow tells you a possible connection, not how often it runs. A debugger trace, by contrast, records a path actually taken under one setup.

Checkpoint: A file is ARM64 PE. Can you select Linux ARM64 and expect it to run? **Answer:** No. The CPU may match, but the executable loader, calling environment, dependencies, and API expectations do not. Static disassembly/decompilation can still be useful.

5. What is inside an executable?

Most executable files contain instructions plus a map describing how to load them. Headers identify the format and CPU. Regions separate code, read-only constants, writable data, metadata, and other content. A file can contain much more than the bytes its CPU will execute.

PE, ELF, managed assemblies, and raw files

PE is the Windows executable image format. Its machine field distinguishes CPU targets, and its section table describes regions. An **RVA** is a virtual address relative to the image base; a file offset is a position in the stored file. They are usually different. The Microsoft PE specification defines those terms and fields.

ELF is used by Linux and other systems. Its header carries class, byte order, machine, and entry information. Program headers describe what the loader needs; section headers organize content for other purposes. The ELF header specification is a useful reference when checking a field. SPECTRA's actual loader support is narrower than the entire ELF format.

A **.NET assembly** commonly uses a PE container but contains managed metadata and intermediate language (IL). Use ILSpy to reconstruct C# from suitable managed content. Ghidra handles supported native machine code. Some files mix managed and native pieces; do not assume one engine explains everything.

Raw firmware or raw code may have no useful executable header. You must supply the range, CPU, byte order, and load address. A raw-mode button is not a detector. It applies your assumptions to bytes.

Imports, exports, symbols, and entry

An **import** describes a dependency requested from elsewhere, such as a library function. An **export** exposes something for other code to use. A **symbol** gives an address a name or other metadata. Symbols may be removed from a release build. A **stripped** binary can still contain code; it simply gives you fewer labels.

The entry point is where a loader transfers control, not necessarily the program's main logic. Startup code can prepare runtime state first. Static linking includes library code inside the file; dynamic linking expects dependencies to be supplied later. “Zero imports” can mean a static binary, incomplete parsing, or a different mechanism. It is not a promise that execution needs no environment.

File bytes versus loaded memory

Suppose a section begins at file offset 0x200 and is loaded at VA 0x140001000. A byte 0x20 bytes into that section is at file offset 0x220 and VA 0x140001020. The relationship applies only inside that mapped, file-backed range. Zero-filled BSS can exist in memory without corresponding file bytes. Do not add an image base to every file offset.

Relocations are records used to adjust address-dependent data when an image loads somewhere else. Position-independent code can accommodate movement in other ways, but a position-independent executable may still need relocations. The intermediate manual watches a benign ARM example in detail. For now, record which address space you mean every time you write a location.

Mini-lab: make an identity card

1. Load a built-in demo and open **File structure**.
2. Record filename, SHA-256, format, CPU, byte order, entry, and region names.
3. Locate a code region and note its file offset and VA separately.
4. Open that file offset in **Hex explorer**. Confirm you are viewing the same region.

Checkpoint: Why can a memory address have no file offset? **Answer:** The loader or program can create zero-filled or allocated memory. A runtime stack, heap buffer, or BSS region need not have stored bytes at an equivalent disk location.

6. Your first lap around SPECTRA

Start with the top-level **Load demo**. This loads Northstar, a synthetic teaching PE. It contains clues chosen for practice; it is not evidence of a real intrusion. If you already have work open, save it first. Our question is simple: “What can I say about this file without executing it?”

The top of the workspace

The file card identifies the active bytes and hash. The summary tiles count extracted strings and heuristic indicator strings, show entropy, and summarize structure. Entropy measures how varied byte values are in a region. Compressed data, encrypted data, and ordinary media can all have high entropy. A low-entropy region can still matter. Treat the byte landscape as a place-finding aid, not a danger meter.

“Analysis complete” at the file level means the basic pass finished. It does not mean every function has been recovered, every engine ran, or every behavior is known. Each engine has its own status. Read those statuses when you change sections.

A useful route through the workbench

Section	Question it helps answer
Strings	What readable clues are stored here?
File structure / Hex explorer	What is this file, and where are the actual bytes?
Detect	What compiler, runtime, packer, or protector fingerprints appear?
Cross-references / Functions	Who uses this clue, and which routine contains that work?
Disassembly / Decompile	What do the instructions and reconstructed logic say?
Debugger lab	What happens under a specific supported setup?
Decoder lab / Recovery lab	What does this representation or recovered byte layer contain?
Binary diff / Firmware files	What changed between versions, or what files are inside this container?
Training ground / Notebook	How do I practice, and how do I preserve my investigation?

Find one clue, then follow it

1. In **Strings**, search for a recognizable path or hostname. Select one result.

2. Read its offset, encoding, length, and tags. Copy its bytes or text only after noting where it came from.
3. Use **View bytes** to see its stored representation. A string row is an interpretation of those bytes.
4. Use **Find code references**. Let code analysis finish. If there are results, open one referring instruction and read nearby instructions too.
5. Save a bookmark or note with the string offset, instruction VA, and the question you want to answer.

No reference is not a dead end. The string might be addressed indirectly, referenced from undecoded code, embedded as data, or unused. Check whether filters and context are hiding results before interpreting an empty view.

One device, one small experiment

On a modest device, begin with small demos and one analysis engine at a time. Save before starting heavier scans. If a worker times out or the browser reloads, reduce the task: one function, one range, one layer. The free-access mission includes knowing how to make progress within a small memory budget.

Notebook entry: “Northstar synthetic demo. Basic identity recorded. Selected string at [offset]. [N] code references found under the current pass. Next: inspect the caller. No execution performed.” Fill in your observations; never copy a count simply because a manual expected it.

The investigation continues after the first result

File structure now includes the address mapping ledger. Debugger lab connects supported routine history with the ordinary registers, memory and stepping workspace, while persistent recording preserves bounded observations separately. Notebook offers selective evidence bundles and a local investigation library. Decompile includes the Android APK/DEX route as well as native and .NET engines. Training ground has six resumable evidence routes, and the public CTF collection supplies longer investigations.

Pick the tool for your next question rather than running every engine because it is there. For a string's location, use Hex and mapping. For its use, follow references and code. For a controlled execution question, prepare the debugger. For tomorrow's work, save the investigation and export the evidence you need.

7. One byte, three addresses: learn the map

You found a string at file offset 0x440. A code view shows 0x140002040. Someone else calls it RVA 0x2040. Nobody has to be wrong: these can be three coordinates for the same bytes. Before following references, name the coordinate system. An address without a coordinate system is like a street number without a street.

File offset: where it lives on disk

A file offset counts from the first byte of the file, which is offset zero. Hex explorer reads those stored bytes. Offset 0x440 means 1,088 bytes from that beginning. It does not imply the computer places the file at memory address 0x440. Headers, alignment and loading rules intervene.

VA and RVA: where the program expects it

A virtual address, or VA, names a location in a process's address space. A PE relative virtual address, or RVA, measures from that image's base. For an image base of 0x140000000, RVA 0x2040 gives VA 0x140002040. The base is an assumption about this image, not a universal Windows constant. A loader can move an image; a runtime map can differ from the preferred header map.

Suppose a PE section begins at file offset 0x400 and RVA 0x2000. The byte 0x40 bytes into that section has file offset 0x440 and RVA 0x2040. The general rule within this section's file-backed span is `RVA = section RVA + file offset - section raw offset`. Do not apply it to an offset outside the section just because the subtraction produces a number.

Walk it in SPECTRA

1. Open a PE and choose **File structure**. Find **Address mapping ledger**.
2. Choose **Declared file headers** as Mapping source. Set Input space to **File offset**, enter a known offset from Strings, and choose **Resolve address**.
3. Read the region name, candidate address and classification. If an original file byte exists, **Inspect original bytes** takes you to it. Verify the same text or bytes.
4. Resolve the reported VA in the other direction. A successful round trip within the same mapping is a consistency check, not independent proof that the program will load there.
5. Choose **Export mapping ledger** and keep it with your note. Write which mapping source you used.

When there is no matching file byte

Programs often reserve memory that starts as zeroes. ELF BSS is a familiar example: a load segment's memory size can exceed its stored file size. A byte in that extra range is real mapped memory, but it has no stored source byte at some later file offset. The correct answer is zero-fill, not a guessed offset. Memory allocated during an experiment can likewise have no file origin.

Conversely, an overlay can be stored at the end of a file without being part of its ordinary loaded image. File-only is useful information. An address can also be unmapped, or match overlapping declared regions. Keep ambiguity visible until other evidence resolves it. Do not pick the first row merely to get a clickable answer.

Raw code and extracted children

For a small raw range, select **Current raw disassembly range**. Its base VA applies to the first byte of the selected range, not automatically the beginning of the whole file. In a range starting at file offset 0x80 and VA 0x6080, a byte at file offset 0x83 maps to VA 0x6083. The flat-image CTF maps explicitly supply these relationships.

After extracting a compressed child, its offset zero belongs to the child. It does not become a simple parent offset by adding the archive header size. Preserve parent hash, member path, extraction description and child hash. Those facts reconnect your investigation without inventing a linear relationship through compression.

Practice and explain

Write three columns in Notebook: file offset, VA and mapping evidence. Fill one row for a known string and one for a zero-filled region, if the file has one. For the second row, use “no stored file byte” where appropriate. The skill is explaining when a conversion is valid, not forcing every cell to contain a number.

If you get stuck: confirm that the active sample is still the intended file. A child investigation or comparison can leave you looking at another artifact. Then check hexadecimal notation, section boundaries, image base and mapping source, in that order. Exact 64-bit address arithmetic in the ledger does not make every such address executable in the debugger.

8. Case file: a Windows PE from bytes to meaning

Question: How does a program announce itself and write data? Open **Debugger lab**, select **x86-64**, and choose **Load Windows demo**. The button loads a benign x64 PE and prepares a modeled session. For this chapter, inspect it statically before pressing Run. The companion copy is `windows-model-x64.exe`.

Build its identity

1. Go to **File structure**. Expect PE32+, x86-64, and three named regions: .text, .data, and .bss.
2. Record the preferred image base 0x140000000 and entry VA 0x140001000. The entry's file offset is 0x200 in this fixture.
3. Open **Hex explorer** at 0x200. These are entry code bytes, not a text message.
4. Find the imports. This fixture names OutputDebugStringA, GetTickCount, Sleep, ExitProcess, CreateFileA, and WriteFile from kernel32.dll.

Those names are clues to expected services. OutputDebugStringA suggests diagnostic output. CreateFileA and WriteFile suggest file activity. They do not tell you whether the calls are reached, what arguments they receive, or what a real system would allow.

Follow the message

1. In **Strings**, search `SPECTRA:`. Expect `SPECTRA: hello from the Windows model`.
2. Select it and use **View bytes**. The message begins at file offset 0x500. Its VA is 0x140002100.
3. Choose **Find code references**. Open the instruction that prepares the address.
4. In **Disassembly**, inspect the following indirect call through an import slot. On this x64 example, the message pointer is prepared in RCX for the first argument.
5. Use the instruction context and import information to connect that call to OutputDebugStringA.

Notice the difference between loading an address and loading its contents. An address-calculation instruction can put a pointer in RCX without reading the string itself. The called service receives that pointer and decides what to do with the bytes.

Follow the file path

Search for `spectra-demo.txt`. The path is stored at file offset 0x700, mapped at VA 0x140002300. Follow its reference and compare the caller's setup with the message's setup. Later code passes a handle returned by CreateFileA to WriteFile. A handle is a value identifying an object in the environment; it is not automatically a file pointer in memory.

Give the entry a modest working name such as `demo_message_and_file` using **Name / annotate**. Include the reason in the comment. A name saves you navigation effort; the comment preserves your evidence. If your hypothesis changes, rename it.

Reconstruct, then check

Send the function to **Decompile**, choose the native Ghidra engine, and use the selected function. Read the output beside the assembly. Indirect calls may look less friendly than a hand-written C program. SPECTRA's current Ghidra bridge does not carry every Windows import-slot type into reconstructed C. Use **Native imports · check indirect calls here** and Cross-references to resolve that context.

Write this conclusion: “This benign PE contains a message and path referenced from entry code. The code prepares calls through named Windows import slots. Static evidence suggests debug output and a file write. Whether those calls complete is the next controlled experiment.”

Checkpoint

Why is “the program created a real file on my computer” unsupported here? **Answer:** Static inspection did not execute it. Even the following debugger lesson uses virtual files inside an explicit model. You must distinguish intended operation, modeled behavior, and actual host activity.

9. Case file: the Linux marker that does not call home

Question: Does a hostname-looking string imply a network connection? Open **Training ground → Load the ELF exercise**. This loads the provided `spectra-first-reference.elf` and searches for its marker. The companion copy is `first-reference.elf`.

Meet the file

The bundled artifact is ELF64, little endian, x86-64, with an entry at 0x401020. It has symbols that help us learn. The entry is startup code; the interesting logic is in `main` and `fixture_helper`. If you compile the included source yourself, addresses and metadata may differ. Use the bundled file for matching observations.

1. In Strings, select `SPECTRA_FIXTURE_ONLY.example`.
2. Record its offset and choose **Find code references**. Open the referring instruction.
3. Find the surrounding function and the call to `fixture_helper`. Use the Functions list if the symbol makes navigation easier.
4. Decompile `main`, then `fixture_helper`. Read how the argument travels between them.
5. Return to Training ground and choose **Check my evidence**. The check looks for the decoded reference; merely finding text is not enough.

Read the source after you investigate

```
static const char marker[] = "SPECTRA_FIXTURE_ONLY.example";
int fixture_helper(const char *s) { return s[0]; }
int main(void) { return fixture_helper(marker); }
```

`const char *s` describes a pointer to character data that this function does not modify through that pointer. `s[0]` selects the first character. The marker begins with S, ASCII 83 decimal or 0x53. Main returns the helper's result. There is no network operation in these two source functions.

Look back at the assembly. You should be able to explain one concrete chain: the caller prepares a pointer, passes it to the helper, the helper reads a byte, and a return value comes back. You do not need to understand every instruction in the entire executable to answer this question.

The zero-caller and zero-import traps

Functions may appear to have no known callers when runtime startup, callbacks, or incomplete recovery supplies the connection. Likewise, a library/import count is only what the parser and file representation expose. Never substitute those counts for reading the relevant path.

This teaching ELF contains ordinary linked startup machinery. It is useful for static RE; it is not the simple syscall-only ELF used in our later process-model lesson. If Debugger preflight reports an interpreter or dependency it cannot provide, stop there and use the dedicated Linux demo. Clearing the warning by pretending the whole file is raw code would discard the very environment the program expects.

Your notebook entry

“The string SPECTRA_FIXTURE_ONLY.example is passed by main to fixture_helper. The helper returns its first byte, 0x53. The inspected path does not use the marker as a network destination. Other executable startup code was outside this narrow question.” Add the actual addresses from your file.

Challenge: If the marker began with A, what would the helper return? **Answer:** ASCII 65, provided that pointer still refers to the changed marker. Changing a note or a display name does not change file bytes. This is a prediction, not a claim that you patched the fixture.

10. Case file: ARM, two numbers, eight bytes

Question: Can you explain a complete function, including its inputs and return? In Training ground, select ARM32 for the hardware exercise and choose **Load the hardware exercise**. The raw file contains eight bytes. It has no ELF or PE header. The lesson supplies little-endian ARM mode and load VA 0x1000.

```
01 00 80 e0   add r0, r0, r1
1e ff 2f e1   bx lr
```

The first instruction adds R1 to R0 and stores the result in R0. The second branches to the address in the link register, LR, which normally holds a return address. This is an entire tiny function, not an operating system. The debugger's prepared return boundary lets the exercise stop when that function returns.

Run it one step at a time

1. Review CPU ARM32, little endian, start 0x1000, and the raw range of eight bytes. Use **Initialize session**.
2. In **Set a register**, set R0 to 2 and R1 to 3. These are your supplied inputs, not values discovered from the file.
3. Choose **Step instruction**. Inspect R0. Expect 5. Read the trace entry connecting the instruction to the changed register.
4. Step again or choose **Run / continue**. Expect a returned status at the lab boundary.
5. Return to Training ground and use **Check my evidence**. The check needs the expected return state, not just an intermediate register value.

Explain it in a higher-level language

Open Decompile with the hardware mapping prepared by the exercise. Use Ghidra, ARM32, little endian, raw offset 0, size 8, load VA and function VA 0x1000. Decompile the function. Its useful meaning is “return left plus right,” regardless of the temporary names chosen by the engine.

A C-style expression might be `return left + right;`. That is not the original source recovered from a time machine. It is a reconstruction of behavior under chosen types and ABI assumptions. For the values 2 and 3, the instruction result and reconstructed expression agree.

Repeat on AArch64 and Thumb

Save your note, select AArch64 in Training ground, and load a fresh hardware exercise. Inputs now use X0 and X1; expect X0 = 5 after a clean return. The raw function is again eight bytes. Thumb's exercise is four bytes and uses R0/R1. Always reload the matching bytes and mode together. Changing the dropdown on ARM bytes does not convert the program into AArch64.

Ordinary 32-bit addition wraps at the register's width. For a second experiment, reload the ARM32 hardware exercise and choose Initialize session again. Set R0 to 0xffffffff and R1 to 1. Predict the low 32-bit result before stepping: zero. This does not prove that a program treats those inputs as signed or unsigned; both interpretations use the same stored bits for this simple addition.

Hardware without a board

This is useful outside malware. Many firmware questions concern small routines: checksums, field extraction, arithmetic, or state transitions. You can learn those routines without booting an entire device. When code depends on hardware registers, interrupts, or timing, the missing environment matters. The intermediate manual explains SPECTRA's explicit ARM MMIO responses.

Checkpoint: What evidence do you now have? **Answer:** Exact raw bytes and mapping, an instruction explanation, inputs you supplied, a completed return, and an observed result of 5. You have not tested every input or proved compatibility with an entire ARM application.

11. Read reconstructed code without getting fooled

Assembly gets close to the CPU; reconstructed C or C# gets closer to how people discuss logic. SPECTRA uses a browser integration of Ghidra for native code and ILSpy for managed code. These upstream projects have broader capabilities than any particular browser integration. Their existence does not mean every desktop feature is present here. See the Ghidra project and ILSpy project for upstream context.

Case file: a counting loop

1. Save your work, open Decompile, and choose **Native demo**.
2. Choose **Ghidra · native / firmware**. Use the entry function at 0x140001000 and choose **Decompile**.
3. Find where the accumulator starts, what condition controls repetition, and which value changes on each iteration.
4. Open the same entry in Disassembly. Compare the zeroing operation, comparison, conditional branch, addition, decrement, and backward branch.

The demo's entry bytes are `31 c0 85 c9 7e 06 01 c8 ff c9 75 fa c3`. Its simple meaning is: start a total at zero; while the signed input is positive, add it to the total and decrease it by one; return the total. For input 3, the arithmetic is 3 + 2 + 1 = 6. For input 0, the total stays zero. Read this as a routine, not as a general Windows program startup.

Names, types, and appearances

Generated names such as `param_1`, `local_8`, or `sub_...` are organizational aids. A variable reused by a compiler may appear as one thing in one view and several temporaries in another. Optimized code can inline functions, remove unused paths, combine arithmetic, or replace branches with other operations.

A type controls interpretation. `char *` suggests a pointer to characters. `uint32_t` is an unsigned 32-bit integer. A `struct` describes fields at offsets in a block of memory. If you declare the wrong type, the output may become easier to read and less correct at the same time. Record why a type is justified.

Where your corrections go

Name / annotate saves an address-based name and comments. **FUNCTION SIGNATURE HYPOTHESIS** in Notebook is a note until you use the controls that pass it to Ghidra. Under **Tell Ghidra the types you know**, an applied signature/type can influence reconstruction. Check the resulting output and warnings after rerunning. A local-variable note does not silently rename every decompiler temporary.

Begin with a short, evidenced name such as `sum_positive_countdown`. Avoid names that assert intent you have not established. Keep the previous interpretation in a note if the correction materially changes your conclusion.

Common confusing output

- **Indirect call:** the target comes from a pointer or slot. Look at imports and cross-references rather than inventing a callee.
- **Unexpected parameters:** the ABI or function boundary may be wrong, or a register may really depend on caller state.
- **Lots of casts:** widths, signedness, or missing types need attention. Some casts express real behavior.
- **Unstructured jumps:** optimization, unusual control flow, or missing edges can prevent a neat loop/if reconstruction.
- **Failure:** check function VA, CPU, mode, byte order, mapping, and supported engine profile before assuming the bytes are protected.

Challenge: Predict the loop's result for 4 and explain it from instructions. **Answer:** 10, from 4 + 3 + 2 + 1. A later debugger experiment can test the prediction; a prediction written in Notebook is not an execution trace.

12. Debugging means controlling a question

A debugger lets you pause execution, inspect state, and decide what to do next. SPECTRA's Debugger lab does this for supported emulated code and explicit process models. It does not attach to a real Windows process, boot Linux, or supply a complete device. You can still answer valuable questions if you keep the experiment small.

Before Initialize

1. Choose **Use detected setup** for an executable and read the load check. For raw code, supply a justified CPU, byte order, range, load VA, and start VA.
2. Choose the environment: **Bare routine**, **Windows process model**, or **Linux ELF process model**. Format and CPU support must both fit.
3. Decide whether you are beginning at program entry or calling a function with prepared arguments. Those require different state.
4. Check dependencies and unsupported features listed by preflight. A passing load check proves a mapping can be prepared, not that the whole run will finish.
5. Initialize, inspect the starting memory/registers, and save a baseline checkpoint before changing anything important.

The controls, in human terms

Control	Use it when	Read the stop reason
Step instruction	You want to see one supported operation's effect	Some architectures combine a branch and delay slot
Step over	You want to pass a call and return to its caller	A fault or breakpoint can stop it early
Step out	You want to return from an observed call frame	Starting mid-function may provide no usable call history
Run / continue	You have a breakpoint or a bounded question	A budget stop is not a normal exit
Pause	You need to stop a running batch	Inspect the last complete state and any deadline warning
Reset session	You want the last initialized setup again	It does not apply newly edited setup fields

Breakpoints and watchpoints

A breakpoint says “pause before reaching this instruction address.” Set **Breakpoint VA** and choose **Add breakpoint**. It is a VA in the loaded session, not a file offset. A conditional breakpoint also checks a supported register comparison. Use it to stop when a loop counter reaches a chosen value; it is not an arbitrary scripting expression.

A watchpoint says “pause when this memory range is read or written.” Under **Watch memory**, select the VA, size, and access kind. A write may already be visible when the pause appears, so inspect the preceding operation. Watching one output buffer is more useful than watching the entire address space.

Registers, memory, and evidence

Read memory uses a memory VA. **Show scratch** and **Show stack** are shortcuts to supplied working areas. **Patch session bytes** changes the emulated copy; Hex explorer still shows the original file. **Set a register** changes your experiment. Record edits so another analyst does not mistake them for naturally occurring state.

The live execution preview retains the latest 200 attempts. Persistent recording can retain more observations within its explicit budget; it must start before the execution you want to preserve. API/model events have their own bounds. Stop near the event you care about, save an experiment, and capture useful memory to Recovery. A register shown after a fault is intermediate state; call it a returned value only after a completed return.

Save and resume an experiment

Under **Saved experiments**, name and save a checkpoint. SPECTRA restores by replaying recorded execution and edits in a fresh emulator, then verifies the result. It needs the same sample and engine build. Importing a checkpoint is not the same action as replaying it. A verification failure means the restore did not establish the expected state.

Checkpoint: Your run reaches its instruction budget. Is the program finished? **Answer:** No. The engine stopped because your work budget ended. Inspect PC, recent operations, and the condition you expected. Increase the budget only when you know what further work will test.

13. Case file: watch the Windows model work

Question: What happens to the message and path from chapter 8 under SPECTRA's Windows model? Save your current file, select x86-64, and use **Load Windows demo**. The prepared benign PE is the same teaching program. Review its environment and load check.

Make a prediction first

From static analysis, expect diagnostic output, a clock query, a short sleep request, a file creation, a write, and an exit. The clock and sleep are modeled behavior; they are not evidence about real elapsed time on Windows. The file path is virtual. Your prediction is specific enough to compare with an event trace.

1. Inspect **Memory map**. Locate the loaded code/data and supplied stack/scratch areas.
2. Save a baseline checkpoint before execution.
3. Step a few instructions. Watch argument registers change before the first imported call.
4. Use **Run / continue** with a modest budget large enough for this tiny demo.
5. Read the status. Expect ExitProcess with zero for this fixture. If your session instead stops early, inspect that reason before accepting the expected result.
6. Open the Windows API trace. Find OutputDebugStringA, CreateFileA, and WriteFile. Record argument/result evidence, including the length requested by the write.

Inspect the captured file

The model captures a virtual path corresponding to `C:\spectra-demo.txt`. Click the captured file to send its bytes to Recovery lab. The fixture writes exactly 37 bytes: the diagnostic message without its stored terminating zero. The source string and written buffer therefore have different boundaries. Check byte length and hex when an exact output matters.

There is no new file at that path on your real device. Downloading recovered bytes is a separate action you choose. This distinction lets the experiment be useful without pretending SPECTRA supplied every Windows subsystem.

What the model adds

The x86/x64 Windows path supplies PE mapping, selected import behavior, TLS handling, and a bounded set of modeled APIs in this release. That can be enough for a focused routine or small fixture. It does not supply a complete Windows kernel, GUI, arbitrary DLL implementations, networking, or a managed CLR debugger. An unknown API stops rather than automatically receiving a fabricated success value.

If a real sample stops on an API, record the name, call site, arguments, and the missing behavior. Your next question may be answerable statically. More advanced experiments can supply an explicit hook, but that hook becomes an assumption and may change the path. The intermediate manual explains how to keep that honest.

Compare the three evidence levels

- **File fact:** the message bytes exist at a recorded file offset.
- **Static relationship:** the entry code prepares pointers and calls named import slots.
- **Modeled observation:** under this session, a virtual file receives the captured bytes and the program exits zero.

Notebook entry: Save the sample hash, environment, unmodified demo setup, exit reason, relevant API events, and captured layer hash. State “virtual file in SPECTRA's model.” That wording lets another researcher understand exactly what was tested.

Challenge: Does a successful model run prove the same binary runs on all Windows versions? **Answer:** No. The experiment verifies a selected modeled path, not real-system compatibility.

14. Case file: Linux reads, writes, and exits

Question: How does an input become output without a full Linux machine? In Debugger lab, choose CPU x86-64 and **Load Linux demo**. This small ELF uses modeled system calls. It is separate from the earlier dynamically linked marker exercise.

System calls in plain language

A system call is a request to the operating system, such as reading bytes or writing output. A real kernel handles that request. In this lesson, SPECTRA's Linux process model handles a supported subset. A file descriptor is a small integer identifying an open stream or file: conventionally 0 is standard input, 1 standard output, and 2 standard error.

The fixture asks to read up to 32 bytes from standard input into scratch memory. It then writes the number actually read to standard output and exits with status zero. Asking for 32 bytes does not mean 32 bytes must be returned. Short reads are normal.

Follow the flow

1. Inspect the prepared recipe. Its standard input hex is `68656c6c6f2066726f6d20746865206c61620a`, meaning “hello from the lab” followed by a newline.
2. Review the ELF process load check, start VA 0x400100, and little-endian x64 mode.
3. Run and read the Linux output. Expect that same line and exit code zero.
4. Inspect the syscall events: read from descriptor 0, write to descriptor 1, exit.
5. Read scratch at 0x60000000. Compare those bytes with input and output. Track the actual returned byte count.

Change one thing

The demo prepares two batch cases: `hello` and `changed input`. Choose **Run cases**. One uses `hello\n`; the other uses `science\n`. Each starts in a fresh emulator. Select a case to inspect its output and stop reason, then choose **Show interactive result** to return to the original session.

This is a repeatable experiment: same binary, same environment, different stdin. An output difference is expected. Whether the executed path changes is a separate observation. The loop-free fixture can process different text along the same instruction path.

Repeat on ARM

Save, select ARM32, and load a fresh Linux demo. The same high-level behavior is implemented with ARM instructions and that architecture's syscall convention. Repeat the input/output checks. You have now separated the meaning of the program from one CPU's instruction spelling.

What ordinary Linux programs may still need

The supported process profiles include little-endian x86, x64, ARM/Thumb, and AArch64 static executable cases; ARM/Thumb also have a bounded static-PIE path. External interpreters, arbitrary shared libraries, unsupported relocations, TLS layouts, sockets, threads, and many system behaviors are not provided. A typical desktop ELF can need these even if it is small.

Missing environment support is a useful result. Write “stopped before a socket operation because this model does not implement networking,” not “the program has no network behavior.” The absence of a supplied feature cannot prove absence in the original program.

Checkpoint: Your input is 50 bytes but the fixture asks for 32 once. What should you expect? **Answer:** At most the first 32 bytes are echoed by this single-read program. That is a program limit in this fixture, not evidence of an encoding failure.

15. Case file: catch a string being built

Question: What if a useful string is absent from static Strings? Programs can decode, decompress, concatenate, or build data while running. The useful bytes may exist only in memory. The built-in decoding exercise gives us a small, transparent example.

1. Save your work. In Training ground choose **Load the decoding exercise**, which opens the debugger's benign decoding demo.
2. Inspect the prepared x64 raw mapping at 0x100000 and scratch area at 0x60000000.
3. Before running, read 64 bytes of scratch and save a baseline checkpoint.
4. Under **Watch memory**, add a write watchpoint starting at 0x60000000 for 64 bytes.
5. Run. When it stops, read the watchpoint event and the instruction that wrote data. Inspect the first changed byte.
6. Remove the watchpoint when you are ready to let the loop finish, then continue to its normal stop. Otherwise it will keep pausing on writes.
7. Read scratch again. Set **Capture bytes** to 64 and choose **Send memory to Recovery lab**.

What you should see

The output contains `https://decoded.example/config`. This is inert teaching text. Do not visit sample URLs as part of RE. The loop's repeating operation uses XOR with key byte 0x5a. XORing the same byte twice with the same key restores the original value, which makes it useful for a small reversible lesson.

In Recovery, inspect the emulated-memory finding. Its provenance identifies the memory address and experiment. Go back to Training ground and use **Check my evidence**. The checker looks for the captured result, not for a URL you pasted into notes.

Why a memory capture needs context

A buffer after one instruction can contain only the first character. A buffer after the loop can contain the whole string plus unused bytes. A different input or key can produce something else. Record when you captured it, how many bytes you captured, and which setup produced it.

A file offset and a memory VA are different provenance. If the output was built in scratch, there may be no identical contiguous string in the original file. Save the source routine VA, input range if known, key evidence, output address, and layer hash together.

Static recovery versus emulation

Recover stack strings in Recovery tracks certain x86/x64 constant-write patterns from decoded instructions. It can recover text written in pieces without running the program. That differs from the debugger's actual modeled instruction execution. Both methods are bounded. Neither is a universal unpacker.

For an unknown sample, first locate a plausible decode routine or buffer and understand its arguments. Starting at a random instruction with empty registers rarely recreates the state a decoder expects. If the routine needs a key from a server that you do not have, say so; a readable-looking guess is not recovered evidence.

Notebook entry: “Under the unmodified decoding-demo setup, the loop wrote the recorded URL into scratch. Captured 64 bytes at VA 0x60000000 after completion. XOR 0x5a is supported by the instructions. No network connection occurred.”

Challenge: Why is the first watchpoint hit not enough to claim the whole string was recovered? **Answer:** It may occur after the first byte write. You must inspect the complete output at an appropriate later state.

16. Rewind a small experiment without losing your place

You stepped past the write that mattered. Instead of guessing which instruction changed the buffer, use **Time travel · routine history** in Debugger lab. This is bounded history for x86/x64 bare routines. It records instruction attempts and analyst edits so SPECTRA can reconstruct earlier boundaries and verify them. It is especially useful for a tiny decoder whose output changes a byte at a time.

First, separate three tools

Tool	The question it answers
Live execution preview	What just happened? The preview retains the latest 200 attempts, not an entire long run.
Persistent recording / portable traces	What observations can I save, search, compare and share? Imported observations do not become a running emulator.
Routine history	What were this supported experiment's registers and memory at an earlier action boundary? Seeking replays from the initial state and verifies the result.

A first rewind

1. Save any current investigation. Load the benign decoding demo described in the previous decoder case. Check x86/x64 and **Bare routine**. Start with a fresh initialization.
2. Before stepping or editing, find **Time travel · routine history**. Choose the 512-action budget and **Start history**. Read the status; do not assume it started if the setup was rejected.
3. Step through the first store, watching the destination buffer. Record the instruction address, destination address and old/new byte.
4. Choose **Step backward**. Read the boundary number and changed state. The store should no longer be part of the inspected past state when you are at the boundary immediately before that store.
5. Choose **Step forward in history** to inspect its result again. These controls move through already captured actions; the ordinary Step instruction control advances the live experiment.
6. Choose **Return to latest**. Your latest experiment has been retained while you inspected the past. Continue from there if needed.

A boundary is a point between actions. An action can be a register edit or an attempted instruction, not only a successfully completed instruction. If you changed a register, the history must explain that intervention too. A fault may have partial effects; the stop reason and state belong together.

Find the earlier access

With a relevant breakpoint or memory-access target configured, **Reverse to breakpoint / access** searches the captured dynamic history. Check the status and resulting boundary before interpreting memory. An access search can land before the matching write, so the previous byte is the expected display. It is not a static search for every instruction that could ever touch the address.

Keep the request small. History has a 2 MiB mapped-memory bound, up to 2,048 actions and 4 MiB of history metadata. Hitting the budget pauses before the next action rather than silently throwing away older history. Individual access capture also has bounds; clipped access evidence is incomplete. A larger budget is not a promise that your phone has spare RAM.

Save the result you actually need

The timeline lives in this tab. A project or evidence bundle does not preserve a complete rewindable history. A selected checkpoint can preserve a state through the supported replay mechanism, but imported checkpoint verification is a separate step. Save the recipe, relevant notes, exported observations and checkpoint before closing. Never write “full recording saved” when all you retained was a checkpoint or short live preview.

Windows/Linux process models and ARM, RISC-V or MIPS history are outside this routine-history mode. Their presence elsewhere in SPECTRA does not extend this feature. If Start history refuses the setup, keep the supported ordinary stepping/recording workflow and read the explanation. You did not fail the lesson by discovering a boundary of the tool.

Small challenge: predict a destination byte before stepping, inspect the write, rewind to the previous boundary, and explain both values in Notebook. Include the input, key and instruction. That is a stronger result than a screenshot of readable output.

17. Case file: a .NET client you can read

Question: Can we find behavior by following managed methods rather than machine-code addresses? Open Decompile and choose **.NET demo**. Select **ILSpy · .NET / C#** and **Browse .NET types**. The benign assembly is ManagedTraining.dll.

Learn the managed vocabulary

An **assembly** packages managed code and metadata. A **namespace** groups related names. A **type** describes something such as a class. A **method** is a function associated with a type. A constructor prepares an instance. Static methods can be called without a particular instance; instance methods can read or change that object's state.

IL is an intermediate language. A .NET runtime normally supplies execution machinery and library behavior. ILSpy reconstructs C# from suitable IL and metadata. Browsing and decompiling those bytes does not run the target's methods, constructors, or DLL entry code.

Start with Classify

1. Search the Types & methods tree for `Classify` in `Spectra.Fixtures.Decoder`.
2. Select it and decompile. Find the conditions for negative, zero, and positive input.
3. Write predicted outputs for -1, 0, and 1. Explain which branch each input takes.
4. Read `Sum` and identify its input array, accumulator, and loop.
5. Read `Decode`. Compare the XOR operation with the native decoding lesson.

The same concept can appear in different representations: a byte-wise XOR loop in C#, C, and assembly. RE becomes easier when you recognize the behavior rather than relying on one exact spelling.

A tiny client with state

Find `LabClient`, its constructors, `SetPrefix`, and `Greet`. The default prefix is “ready.” Greet joins the prefix, a colon, and the supplied name. Predict Greet("Ada") after default construction: `ready:Ada`. After SetPrefix("hello"), the prediction becomes `hello:Ada`. You have described an order-dependent experiment without executing it.

Now inspect `Fingerprint`, `Credentials`, `Json`, and `FetchAsync`. They offer recognizable cryptography, credential-object, serialization, and network-related calls. Compare labels with method bodies. `MisleadingNameAuth` is a useful reminder: a name can sound security-related while its actual work is ordinary text.

Dependencies and labels

If a real client refers to another assembly, **Add local .NET dependencies** can supply matching metadata. Resolution uses full identity, including version and public-key token. Adding an arbitrary DLL with a similar filename is not a repair. The current limits are 32 files, 16 MiB each, and 64 MiB including the main image.

Crypto/Auth/Network/Serialization/Filesystem labels are leads with evidence, not verdicts. Read the relevant reference or call. A SHA-256 helper does not prove encryption; a NetworkCredential object does not prove a successful login.

Harness lab: know the boundary

You can select methods/classes and download a C# harness, or export selection and clues. This prepares an external experiment; it does not run managed code in the browser. The runner needs a compatible external runtime and original dependencies. Its execution mode can invoke real code with the host's permissions. For this beginner/browser path, inspect the generated source and recipe only. No external execution is required to finish the chapter.

Checkpoint: What is missing from a prediction of Greet's return? **Answer:** An observed execution under specified object state. Static reasoning is still useful, but label it as a prediction until tested. The intermediate book develops the method-selection and recipe workflow further.

18. Case file: an Android package, from manifest to method bytes

An APK is a package, not one long block of ARM instructions. It can contain managed Dalvik code in DEX files, a compiled manifest, resources, assets and native libraries for particular CPUs. Your first job is to identify which kind of evidence you are looking at. SPECTRA's Android workbench reads these pieces as local data; you do not install the APK.

Start with the harmless fixture

1. Save your current work and go to **Training ground → Load the Android exercise**. The **Android demo** in Decompile opens the same fixture. The companion practice pack also contains `AndroidTraining.apk`.
2. In Decompile choose **dex2jar + Vineflower**, then **Read APK / DEX**. Allow the local engine to initialize and wait for the actual status.
3. Record the package hash, number of DEX files and class identities. Expect two root DEX files: `classes.dex` and `classes2.dex`, containing Training and SecondDex.
4. Keep the package open while working through the next sections. Opening a DEX or native library as a child replaces the active sample, so save before taking that route.

This fixture is deliberately unsigned and not installable. That is useful: the investigation does not depend on a phone installer or an Android account. The specimen teaches structure and reasoning, rather than impersonating a complete mobile app.

Read the declared capabilities

Expand **Manifest, resources & package files**. Select `AndroidManifest.xml` and choose **Decode XML / resource table**. A compiled Android manifest is not necessarily plain text, which explains why a raw Strings pass can miss its readable structure. Record package identity and the INTERNET permission.

A manifest declares components and requested permissions; Android's manifest overview explains that contract. A permission declaration is not a packet capture or proof that an app used the permission. In this fixture, the teaching code makes no network calls. Write “declares INTERNET,” then state what additional code or runtime evidence would be needed for a network claim.

Follow a call across DEX files

1. Find and select **SecondDex**, then choose **Decompile**. Read the reconstructed Java and locate `total`.
2. Identify its call to `Training.sum`. Write down the argument relationship instead of just the method's name.

3. Expand **Source references** and follow the unambiguous definition into Training in the other DEX file.
4. Read `sum`. Describe how its arithmetic relates to the caller's return value. A method boundary does not make the value flow disappear.
5. Return to a method selection and choose **Inspect method bytes**. Record the original DEX filename, DEX hash, method descriptor, code offset and instruction length.

A descriptor identifies a method's parameter and return types. It matters when two methods share a name or an app has overloaded methods. The DEX filename matters when multiple DEX files have overlapping identities. Keep both instead of assuming that a short name uniquely identifies code.

SPECTRA converts DEX to JVM class bytes with dex2jar and reconstructs Java with Vineflower. The output is an explanation recovered through those engines, not the developer's original source. Even if an app was written in Kotlin, this route produces Java. Generated constructs and unusual casts can reflect compilation and missing metadata; inspect warnings before concluding they are deliberate obfuscation.

Put the source back beside evidence

The method preview shows at most 256 original instruction bytes. DEX is a defined bytecode format; it is not native ARM machine code simply because Android devices often use ARM. The AOSP DEX specification defines its layout and types. Do not select ARM in native disassembly and treat the resulting accidental instruction spellings as a meaningful decompilation of DEX.

Source-reference links are symbolic relationships reported by the engine. They do not promise a one-to-one mapping between a Java line and a DEX instruction. For a report, identify the method and bytes independently of the presentation line number. **Export source** saves the selected class; **Export evidence** retains its context and warnings; **Export inventory** describes the package and class identities.

Resources and native code are separate threads

Select `resources.arsc` and decode it. Inspect IDs, names, values and variants. A numeric resource reference may remain numeric. The view does not rebuild a complete Android project or decide which variant a particular phone would select. Treat a readable URL or label as stored data until you find its consumer.

The package also contains a harmless native x86-64 addition library. Save the APK investigation before **Open child investigation**. The native ELF belongs in the ordinary Ghidra workflow with its own CPU and mapping. Preserve the parent package hash, member path and child hash. A compressed package member does not have an invented continuous parent-file address for each decompressed byte.

Finish the learning route

In Notebook, write one declared capability, one demonstrated source relationship and one original DEX evidence location. Prepare a findings inventory, return to Training ground and choose **Check my evidence**. Read any missing requirements rather than repeatedly pressing the button. Choose **Save lesson workspace** after the checks reflect your observations.

This route does not verify APK publisher signatures, install/rebuild/sign applications, merge split sets, emulate Android or debug ART. Packed or dynamically obtained DEX may be absent from the package you have. Missing classes can explain incomplete results. The useful next question is “which layer is missing?” rather than “why didn't the decompiler recover an entire phone?”

19. Encoded, compressed, packed, encrypted: choose the right question

When text is missing, resist the urge to throw transforms at every byte. First ask what representation the code expects. Base64 text, compressed bytes, a packed executable, and ciphertext are different problems. Your next step should follow evidence from a header, instruction sequence, API, or surrounding layout.

Use Detect for leads

1. Open **Detect** and choose **Scan with DiE**. Leave Heuristics at its default for the first pass.
2. Read the category, name, offset/region, and whether a result is database-based or heuristic.
3. Open **What does this tell me?** and the **Raw DiE report**.
4. Save the report and pick one claim to check in structure or code.

Detect It Easy identifies file/compiler/packer-related patterns; its upstream project describes its detection approach. SPECTRA pins a particular engine/database release. A packer label is a lead about wrapping, not automatic unpacking. No match does not prove there is no obfuscation. Deep scan looks further for evidence; it does not remove arbitrary protection.

Two small recovery experiments

Open companion `base64-message.txt`. In Recovery select Original sample, offset 0, length 8, and Base64 under **Decode bytes with a known format or key**. Choose **Decode selected bytes**. Expect a five-byte layer containing hello. Inspect its hash, source range, and strings.

Now save and open `xor-message.bin`. Its bytes are `32 3f 36 36 35`. Select offset 0, length 5, Repeating XOR, key hex `5a`. Decode and expect hello again. Unlike the earlier Decoder scratch output, this layer preserves a byte-backed source chain. Export or save it with the investigation.

Hunting versus knowing

Hunt encoded strings searches a bounded range for candidate encodings and one-byte XOR text. A candidate can be accidental. Check whether the entire region decodes coherently and whether instructions use that transform/key. The hunt scans only its documented bounded portion; no results is not proof that there are no hidden strings.

A known transform is stronger when code supplies its parameters. For multi-byte XOR, selecting the wrong starting byte changes the key phase. For compression, choosing the

wrong wrapper can make a valid stream fail. For cryptography, the key, nonce/IV, range, mode, tag, and any additional authenticated data can all matter.

What the controls mean

Transform	What you must know
Base64 / Hex text	The selected bytes are text in that representation
Gzip / Zlib / Raw DEFLATE	Which wrapper surrounds the compressed stream
Repeating XOR / RC4	Correct key and stream/range assumptions
AES-CBC	Correct AES key, 16-byte IV, and the expected padding/layout
AES-GCM	Correct key, 12-byte nonce, optional AAD, and trailing authentication tag

Those cryptographic controls do not recover arbitrary keys. A failed authentication check is evidence that the supplied parameters/data do not verify, not something to ignore. The intermediate manual explains how to preserve parameter provenance.

Layers have their own offsets

If you decode bytes starting at original offset 0x900, offset zero in the new layer means the first decoded byte. It is not automatically original offset zero or 0x900 plus the layer offset. Compression can make that relationship non-linear. Use **Use as next layer** only when the next transform has a reason; preserve the chain.

Checkpoint: A high-entropy region decodes into two readable words under one XOR key. Have you found the configuration? **Answer:** Not yet. Seek coherent structure, broader plaintext, key/range evidence, and a consumer in code.

20. Case file: a firmware package with another program inside

Question: How do you investigate something that contains other things? Open companion `fixture-firmware.tar.gz`. This is a synthetic archive made for the book, not a bootable device image. It contains a text configuration, a README, and the tiny ARM addition routine you already understand.

Outer wrapper, inner content

A firmware download may include vendor headers, compressed filesystems, boot code, applications, and configuration. The outer filename does not tell you the CPU of every member. A package can contain code for several components. First identify the container, then identify each interesting child on its own terms.

1. Open **Firmware files**. Set **Container file offset** to 0 and choose **Inspect container**.
2. Expect a gzip wrapper around a tar archive. Review the entry names, kinds, sizes, and offset provenance.
3. Find `etc/device.conf`. Use **Download bytes** if you want a separate copy, or open it as a child after saving the parent investigation.
4. Read the configuration in Strings/Hex. It contains model=training, port=8080, and a read-only mode value.
5. Restore/open the parent again, inspect the archive, and select `bin/add-arm.bin`. Choose **Open child investigation**.
6. Check the child Notebook for parent/member provenance. Configure the child's raw ARM32, little-endian, eight-byte range at VA 0x1000 and repeat the arithmetic investigation.

What did the configuration prove?

A setting named port with value 8080 is stored configuration. It does not prove a listening server starts, which interface it binds, or whether a program actually reads the setting. To establish use, find the consuming executable and follow its parser or references. In this tiny synthetic package, the ARM function does not read that configuration; the separation is intentional.

This is an important firmware habit. Adjacent files are not automatically connected. A service name, certificate, or default setting is a lead. Identify the startup or consuming path before drawing a system-level conclusion.

Offsets after extraction

A tar member offset inside decompressed data is not the same as its position in the compressed download. A SquashFS file may span compressed blocks and fragments. Keep container format, wrapper, parent hash, member path, extraction metadata, and child hash together. A child file can then be discussed reproducibly even when no simple original-file offset describes all its bytes.

Supported containers

This release supports selected stored/deflated ZIP, USTAR/PAX tar, newc/crc CPIO, gzip wrappers, and little-endian SquashFS 4 using gzip, XZ, or Zstandard. Unsupported versions/compressors stop explicitly. Limits include 64 MiB expanded data, 16 MiB per regular member, 2,000 entries, and a 30-second job. Hitting a bound does not prove corruption.

Links, devices, and other special entries remain metadata. SPECTRA does not follow a symlink into your computer or create a device node. Extraction gives you inspectable bytes; it does not boot the firmware or execute its programs.

Checkpoint: The archive contains a setting called password. What should you do next?
Answer: Preserve the context, determine whether it is a placeholder/default/actual value, and find the consumer. Do not publish possible credentials or test them against a live device without an authorized research plan.

21. Case file: one changed instruction, two program versions

Question: Did a rebuild change the function's arithmetic? Open companion `add-one.exe` as the main sample. In **Binary diff**, choose **Open second binary** and select `add-two.exe`, then **Compare**. These are benign synthetic PE routine fixtures; they are not intended as complete Windows applications.

Confirm the comparison before interpreting it

1. Check before/after filenames and hashes. Both are x64 PE fixtures with an entry at 0x140001000.
2. Review changed strings/imports as well as matched functions. This pair changes one entry-code byte while retaining the surrounding fixture data.
3. Select the entry-function pair. The old routine's bytes at file offset 0x200 are `8d 41 01 c3`; the new routine's are `8d 41 02 c3`.
4. Compare assembly. The address-calculation instruction computes a result from ECX plus one or plus two, placing it in EAX. Here it is used as arithmetic, not to read memory.
5. Choose **Decompile selected pair** and compare the reconstructed constants.

If the entry is in an unmatched/ambiguous group rather than a confident pair, use the known entry on each side and compare manually. A matcher is an assistant, not the authority that decides whether you may investigate two functions.

Predict and test a focused call

For input 7, predict old result 8 and new result 9. To test, open each file separately in Debugger lab, use detected x64 Windows mapping, set start to the entry, and enable **Call the selected entry as a native function**. Choose Windows x64 ABI and arguments `["7"]`, initialize, and run. This function-entry experiment supplies a return boundary; it does not pretend the synthetic entry is a full process startup.

Check for a completed return and read RAX/EAX's low 32-bit result. Record both sample hashes and setups. A return value after a fault would not validate the prediction.

Why this matters for patch review

Real updates can change checks, constants, error paths, dependencies, or data without changing every function. A new string such as “invalid length” can point toward a patch, but the check's location and covered paths matter. This tiny pair teaches the first move: explain a specific difference rather than announcing that one file is newer or safer.

Exact decoded-byte matches mean equality of the compared instruction bytes under the recovered boundaries. They do not prove full behavioral equivalence: data or a callee might

change. Shape/name matches are candidates. Unmatched functions may reflect inlining, layout changes, missing recovery, or actual additions/removals.

A useful report sentence

“At entry VA 0x140001000, the immediate addend changes from 1 to 2. The routine returns 8 versus 9 for supplied input 7 under the same Windows x64 function harness. This pair does not contain or demonstrate a security fix.” That last sentence matters: we are practicing a method, not inventing a vulnerability.

Challenge: If all code bytes matched but a lookup table changed, could behavior change? **Answer:** Yes. Code can consume different data. Include data, imports, and environment in the investigation.

Compare data even when there are no functions

After **Open second binary → Compare**, inspect **Raw byte comparison** above the matched-function results. It gives absolute file offsets and before/after bytes without needing disassembly. A dash means one file ended before that position. This is ideal for the CTF's configuration captures and tiny flat images.

Record which file is the main/before file. Opening a comparison file does not make it the main sample in Hex explorer or Disassembly. Switch the main file explicitly when you need to decode its revised code. This raw comparison is positional: an inserted byte can shift everything after it and produce many differences. It does not automatically align relocated data or prove which differences affect behavior.

22. Your first real malware investigation

The harmless cases teach habits you can carry to real samples. Real malware adds uncertainty and consequences, not a new set of magic buttons. Start with static inspection and one narrow question. No part of this chapter requires launching a sample on your device.

Where to get a sample

MalwareBazaar is a research sample database. Its browser supports searches such as `sha256:...`, `file_type:elf`, and tags. Record the sample page, exact hash, type, and metadata before downloading. Follow the current download/authentication instructions on the service. Labels are community/engine context, not conclusions from your own analysis. See its FAQ for service details.

If you use a file already collected by an authorized honeypot, preserve its collection time, original hash, and the relevant existing log context. This book does not walk through deploying a honeypot. A downloaded file and a shell command in a log are data to inspect, not instructions for your own terminal.

Keep the file inert

Use the file picker inside SPECTRA to open a sample as data. Do not double-click it to “see what happens,” run its installer, enable document macros, or follow URLs found inside it. Keep downloaded unknown files apart from ordinary documents. A current browser and local analysis reduce some exposure; they do not make hostile parsers or files mathematically risk-free.

Sample downloads may arrive in password-protected archives. Follow the archive's current instructions using an appropriate trusted archive tool; SPECTRA's container inspector does not decrypt encrypted ZIP archives. Do not disable device protections to force a download or run. If your phone/tablet cannot extract the archive safely, continue with the benign lessons and the sample's public metadata. There is no learning requirement to execute live malware.

A complete first-pass worksheet

1. **Identity:** Open the extracted member. Compare its SHA-256 with the expected member hash. Record format, CPU, size, entry, and any parse warnings. An archive hash is not the member hash.
2. **Question:** Choose one: where is configuration stored, what file path is prepared, or which routine decodes a buffer? Avoid beginning with “identify every capability.”
3. **Structure:** Review executable and data regions, imports/symbols, suspicious-looking boundaries, and whether a managed header points toward ILSpy.

4. **Detection:** Run DiE if within its cap. Keep the raw report and distinguish heuristic from database evidence.
5. **Strings:** Search a small theme: configuration, paths, error messages, or protocol terms. Record exact offsets and encodings. Do not open extracted network indicators.
6. **Context:** Follow a reference into Disassembly. Find the surrounding function and its caller. Note unresolved targets and analysis limits.
7. **Reconstruction:** Decompile the selected routine. Check the operations supporting your hypothesis against assembly or IL.
8. **Recovery:** If code establishes an encoding/key, decode the exact relevant range and preserve provenance. Do not treat an attractive guess as confirmed plaintext.
9. **Optional model:** Read debugger preflight before execution. Use a focused supported setup only when you understand inputs and missing state. A static-only finding is a valid result.
10. **Report:** Export the project/evidence and write what is known, suspected, and unresolved.

Three platform routes

Windows native PE: use imports, strings, call arguments, native reconstruction, and supported x86/x64 model experiments. A Windows ARM64 PE can be statically analyzed with the appropriate CPU while remaining incompatible with the x86/x64 Windows process model.

Linux ELF: inspect machine, class, byte order, program headers, interpreter/dependencies, symbols, and syscall-related code. An ARM ELF can be statically inspected even when its startup requires unsupported behavior. Zero imports does not equal no dependencies.

Managed .NET: browse types/methods, follow data from entry or a meaningful caller, inspect serialization/crypto/network evidence, and resolve matching local metadata dependencies. Do not select native machine-code emulation as a substitute for a CLR managed debugger.

What a first report can honestly say

“Sample [hash], collected from [source]. The selected function references a file path and passes a buffer to a named write operation. The buffer appears to be produced by this decode routine. The decoder's output was [statically recovered / observed under stated emulation]. Network behavior and persistence were not established by this investigation.” Replace every bracket with evidence or a stated unknown.

Checkpoint: Your chosen sample is packed and the current tools do not expose its inner code. Have you failed? **Answer:** No. You can document the wrapper, evidence, affected regions, and exact limit reached. Unsupported does not mean harmless, and it does not justify inventing an unpacked result.

23. Keep your work so tomorrow starts ahead

An investigation becomes valuable when understanding survives the tab. Notebook stores notes and names, but several save/export controls have different purposes. Know what each keeps before closing your browser or sending a file to someone else.

Notes worth writing

```
Question:
Sample name and SHA-256:
Format / CPU / byte order:
Observed bytes and locations:
References and caller:
Hypothesis:
Experiment inputs and assumptions:
Stop reason and result:
Alternative explanations:
Next useful step:
```

Use both file offsets and VAs where relevant and label them explicitly. Keep captured layer hashes and parent-child relationships. If you changed a register, supplied a hook, or patched session memory, put it in the experiment record. A screenshot helps communicate, but structured settings and exact bytes make it reproducible.

Save a full investigation

1. In Notebook, give the investigation a meaningful name. A collection can group related firmware versions or a client and its libraries.
2. Choose **Save new investigation**, or select an existing investigation and **Save revision to selected investigation**.
3. For a portable backup, choose **Save project**. Decide whether to include original sample bytes.
4. Also export specialist reports when useful: code JSON, recovered bytes/report, decompilation evidence, batch report, or file inventory.
5. Reopen one saved project as a practice exercise after preserving your current work. Verify the sample identity and notes before relying on the workflow.

Local storage is not a vault

The investigation library includes original and recovered bytes. The current limits are 32 investigations, up to eight retained revisions each, and 512 MiB total, subject to a lower browser quota. A ninth revision replaces the oldest. Clearing site data removes local saves. Private browsing, storage pressure, and device cleanup can also affect persistence.

Auto-save the full workspace and **Save full workspace on device** keep local state, including bytes. Wait for a saved status. **Resume saved workspace** restores that workspace; it does not turn browser storage into a portable backup.

Before you share an export

A full project can contain a live sample, recovered data, dependency assemblies, the second comparison binary, virtual files, or supplied secrets. Omitting the main sample does not guarantee every other byte artifact is absent. Inspect what you intend to share. Use a narrow report when someone needs findings rather than the entire investigation.

Recovery reports omit manually supplied transform keys, while a derived one-byte XOR key may be included as evidence. If reproduction requires a key, record it deliberately in an appropriate private note rather than assuming every export retained it.

Restoring experiments

Project import restores data. Replaying an emulator state is a separate execution action using the matching engine/sample. Checkpoints are not interchangeable with a general VM snapshot. Keep the corresponding offline release if long-term replay matters. If a restore fails verification, retain the failure and settings rather than editing fingerprints to make it appear compatible.

Checkpoint: Is Export findings the same as Save project? **Answer:** No. A findings export serves a narrower report purpose; the full project preserves substantially more investigation state. Choose based on what you need to reopen or share.

Choose what leaves your investigation

Selective evidence bundle in Notebook is the route for a deliberate handoff. Start with the findings-only default. The nine evidence groups let you choose notes, analysis, recipes, traces, lesson progress and separately controlled byte categories. Original, recovered, dependency and comparison files have separate inclusion choices; including one does not mean all are included.

1. Select the groups needed for your question. Read the labels and check whether notes, inputs or traces contain private values.
2. Choose **Prepare inclusion inventory**. Inspect every listed member, size and SHA-256. This is an immutable prepared snapshot.
3. If you change your work or selections, prepare again. An old inventory cannot include a new note you wrote afterward.
4. Choose **Download prepared bundle**, wait for completion and save the completed file where your device can retain it. Interrupted output is not a verified finished bundle.
5. To inspect a received bundle, choose **Open evidence bundle**. Review the verified member list first. **Replace workspace with reviewed evidence** is a separate, explicit action: save your existing workspace before replacing it.

If original bytes were omitted, restoration may require the exact matching sample. Restoring a project does not replay it. A bundle hash validates members against its manifest; it does not make an unknown author's conclusions authoritative. Group selection is also not automatic redaction: a note can contain a credential even when you excluded original sample bytes.

Save more than one investigation

The **Investigation library** gives each binary a name and optional collection. Use **Save new investigation** for a new item and **Save revision to selected investigation** for progress on the selected one. Check the selection before overwriting your idea of “latest.” The library keeps up to eight revisions per investigation; a ninth replaces the oldest. It includes original and recovered bytes and is local to this browser.

The library's ceiling is 32 investigations and 512 MiB total, subject to lower browser storage availability. Use a collection for related firmware versions or client libraries. Export a portable copy for anything you cannot afford to lose. Clearing site data removes the library; signing into another website does not back it up.

Resume a built-in lesson

The six built-in routes cover native code, decoding, hardware, Windows, Linux and Android. They bind checks to an exact fixture hash and lesson version. Perform the requested observation, then choose **Check my evidence**. A missing-check message tells you which evidence has not been captured. Merely pasting a decoded sentence into Notebook is not a newly observed recovery capture.

Use **Save lesson workspace** before leaving and **Resume saved workspace** when returning. Portable projects and selected lesson evidence in bundles also retain progress. Imported or restored milestones can be historical: they record previous work without pretending it was just independently repeated. The checklist checks evidence presence; it does not grade every interpretation you wrote.

On an iPad or phone, keep one heavy engine job active at a time and save before switching away for a long stretch. Touch and keyboard routes use the same evidence model. A smaller screen does not change what a hash, range or result means; a discarded tab does change which unsaved work survives.

24. Save a run, inspect a trace, and know what survived

A trace is an ordered record of observations. It can help you answer which instruction ran before a write, which modeled call returned an error, or how two supplied inputs differed. Its value depends on what was recorded, when capture began and what was left out. A convincing-looking list does not automatically describe the whole execution.

Record the Linux exercise

1. Save your current investigation. Load the built-in Linux exercise and inspect its virtual stdin, architecture and process-model preflight.
2. Initialize the session. Before stepping or running, find **Persistent recording**. Start with the 2 MiB byte budget for this small exercise and choose **Start recording**.
3. Run the prepared routine to its expected exit. Inspect the read/write/exit events, output bytes and stop reason. A recording cannot repair missing OS behavior; a stopped unsupported syscall remains a stopped experiment.
4. Choose **Stop recording** and wait for the saved/completed status. This flushes the remaining segment. Open the recording for review and inspect its first and last records.
5. Export the recording, then use **Save completed file** when the prepared download is available. Confirm the file appears in your device's downloads. Preparing an export is not the same as saving it outside the tab.

Persistent recordings use ordered segments in local browser storage. The chosen byte budget and record/instruction limits bound the recording. They do not promise infinite execution or zero memory use. A killed tab can lose the latest unfinished segment. Browser storage can also be cleared or evicted; an exported file is the portable copy.

Read an observation carefully

For a write, record destination, captured bytes and order. For a register, distinguish a value recorded before an attempt from a final state reported elsewhere. For a model event, name the modeled service and supplied response. A fake virtual file in an exercise is evidence of that experiment, not a disk write on your real device.

The long recording export is newline-delimited JSON: a sequence of records with a required completion footer. A missing footer or interrupted-export status matters. Do not rename a partial download to make it look finished. Retain a completed earlier export or repeat the bounded benign experiment when appropriate. Gaps must remain gaps, not guesses.

Open somebody else's trace

SPECTRA offers two read-only trace slots, search and retained-order comparison. Its compact version-1 trace file contains the latest 200 instruction records plus bounded event/write channels and final mappings. It is not the same as the longer version-2 persistent recording. Check the format and completeness before assuming the start of the file is the start of a process.

Imported observations stay unverified. A valid checksum detects alteration relative to the stored digest; an author could recompute both. It does not establish who recorded the trace or whether their story is true. The import does not initialize an emulator, execute a sample, or turn a record into a time-travel checkpoint.

If the sample hash matches, byte/assembly navigation can connect recorded locations to your open artifact. If it does not match, keep the mismatch visible. The same address in another build can refer to different code. External adapters accept documented subsets, including uncompressed x64dbg v1 traces and bounded Chrome trace JSON, but adapter output is still imported evidence with its original scope.

Compare events, not just row numbers

One run may perform an extra check before reaching a common event. An event-aligned comparison can help find the later correspondence; row 20 versus row 20 need not be the same action. Alignment is bounded and heuristic, so inspect what it matched and the unmatched spans. A matched event label does not prove equal inputs, equal memory or equivalent behavior.

Practice: record the Linux exercise with its ordinary virtual stdin, then repeat with a short different input under the same setup. Compare the returned byte counts and stdout. Keep both inputs, hashes and recording identities. Explain whether a difference came from the input, environment or code. If one run is incomplete, say so.

25. Enter the CTFs: solve a question, not a guessing game

A capture-the-flag challenge gives you a bounded question and a way to check an answer. In these SPECTRA CTFs, the real prize is learning how to explain a binary. A flag is a small answer string in a specified format; it is not a command to execute and not a substitute for evidence.

Open the SPECTRA CTF collection. This edition covers eight campaigns and fifty-seven stages. The beginner route is The Dead Channel, Glass Relay and Cold Storage. The intermediate book continues with Ghost Circuit, Hollow Signal, Iron Veil, Black Loom and Puppet Core. The public casebooks and benign files are enough to study in a browser; an account is not needed to learn the techniques.

Your first case folder

1. Choose one campaign and read its FIELD-MANUAL before opening random files. The companion pack contains the same public files under `practice/ctf/001` through `006`.
2. Compare the downloaded file's hash with the campaign's SHA256SUMS or the companion manifest. Record the exact filename and hash. A hash helps you know which bytes the walkthrough describes.
3. Open the stated file using SPECTRA's **Open file**. Never launch a specimen as a native program. The tiny raw files often have no executable header; the supplied map tells you how to inspect them.
4. Keep Notebook open for the question, observed locations, reasoning and next step. Save before replacing the main file or decoding another range.

How to use the walkthroughs

For each stage, first read the question and predict where the answer could live. Follow the numbered actions, then compare your observations with the explanation. Use the hints one at a time. Before submitting, write one sentence explaining why the answer follows. If you can only say “the hint said so,” repeat the observation in Hex or Disassembly.

Flags use literal formatting rules. A required file offset is not a VA; decimal digits are not always interchangeable with padded hexadecimal; a loaded field is not a later calculated return. The stage's format tells you how to express the result, while the bytes and code determine the result itself.

Learning progress versus the CTF checker

Training ground's saved evidence checklists are local SPECTRA learning routes. The Church Discord CTF menu is a separate system: use `/spectra`, select a campaign, or specify `ctf:001` through `ctf:006` where the command offers that option. Its private stage view provides hints and a flag-submission form. Saved CTF completion and patch collection belong to that account and campaign, not to your browser's lesson workspace.

You can work through every method here without submitting a flag or collecting a reward. If you use the checker, an accepted answer means the answer matched; it does not grade your written reasoning, assumptions or proof. Save those in Notebook and export your investigation.

Raw-range controls that often trip people up

In **Disassembly → Raw range / architecture override**, enter FILE OFFSET, BYTE LENGTH, BASE VA, ARCHITECTURE and BYTE ORDER. Choose **Decode range**, then **All decoded instructions** when the case asks for the whole snippet. Hex values need `0x`. Decimal 19 and hex 0x19 are different lengths.

Decoding another raw range replaces the current code results. Keep earlier notes and map rows so you can return. Raw decoding does not create OS state or prove that data is code. Padding bytes can spell valid instructions by coincidence. The supplied CTF CODE rows identify deliberate routines; DATA rows should be read as data.

When stuck, check the active filename, offset versus VA, raw length, CPU and endianness before changing your theory. On touch devices, the same labels may stack vertically; scroll to the full control group. A hidden column or narrow screen does not change the required bytes.

26. The Dead Channel: your first five evidence trails

A small file can teach a whole investigation. This campaign moves from direct text to encodings, a tiny branch and a changed opcode. Work in order; each stage adds one idea. You only need the file picker, Strings, Hex explorer, Decoder lab, Disassembly, Binary diff and Notebook.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/001/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · Carrier signal	Strings + evidence
02 · Wrong end of the wire	Hex + byte order
03 · Wrapped transmission	Decoder lab
04 · The gate	Disassembly + branches
05 · One-byte betrayal	Binary diff + behavior

Stage 01 · Carrier signal

The question. A dead relay left a small memory record. Recover the operator flag without executing anything. Record the flag and its file offset in your SPECTRA Notebook.

Open these files: 01-carrier.bin.

1. Download 01-carrier.bin. Open SPECTRA, choose Open file, then select the downloaded file.
2. Open Strings. Start with ASCII only and a minimum length of 4. Filter for SPECTRA.
3. Select the matching row. Read its file offset and use View in hex to verify the surrounding bytes.

Read what happened

A string extractor finds runs of bytes that look like text. It does not decide whether a string is used, true or important. Start with a short ASCII minimum so the deliberately stored token is visible, then filter. If you leave a previous filter or encoding selected, you can hide it yourself. In Hex explorer verify the bytes beginning at file offset 0x140 and the terminating zero. Record the offset of the first character, not the terminator or the row's left-hand display boundary.

The surrounding bytes matter because a search result can be a preview. Read the complete bounded string before transcribing it. This stage demonstrates direct stored evidence, not decryption or execution. Your note should say the file contains this string at this offset; it should not claim a remote operator or infection merely because the scenario uses a relay story.

Hints, one at a time

1. A .bin extension does not mean the file must be run. Start with printable strings.
2. The correct string begins with SPECTRA{. Search that literal prefix.
3. Inspect file offset 0x140. The flag ends before the zero byte.

Response format: `SPECTRA{the_text_you_find}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 02 · Wrong end of the wire

The question. The directory points to a message that ordinary ASCII extraction misses. Parse the header, follow the offset, and recover the text. Header: 0x00 magic (4 bytes); 0x04 version (u16 LE); 0x06 record count (u16 LE); 0x08 payload offset (u32 LE); 0x0c payload byte length (u16 LE); 0x0e encoding tag (1 = UTF-16LE, 2 = UTF-16BE). Offsets are absolute file offsets.

Open these files: 02-directory.bin.

1. Open 02-directory.bin in Hex explorer. Read the directory fields using the supplied layout.
2. Jump to the payload offset and inspect exactly the declared number of bytes.
3. In Strings, change Encoding to All encodings or UTF-16 BE. Correlate the recovered string with the directory.

Read what happened

The directory header and its pointed-to text use different byte orders. This is legal: an integer field's endianness does not force every text encoding in the file to match it. Read the four offset bytes as a little-endian integer. Bytes 80 01 00 00 point to file offset 0x180. Then use the encoding tag and length to decide how many text bytes to interpret.

The selected text is UTF-16 big endian: in simple ASCII-range characters, a zero byte precedes the character byte. ASCII extraction may show fragments or nothing. View the exact counted range in Hex and interpret paired bytes, preserving the distinction between a character count and byte count stated in the header. Never reverse the character order as a substitute for decoding endianness. This is the same habit you need for file-format directories and resource tables.

Hints, one at a time

1. LE describes the integer fields. It does not necessarily describe the text.
2. The payload-offset bytes are 80 01 00 00. Interpret them least significant byte first.
3. The encoding tag is 02. Look for UTF-16 BE text at 0x180.

Response format: `SPECTRA{the_decoded_text}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 03 · Wrapped transmission

The question. The relay wrapped a message for transport and obscured its bytes. Its configuration survived in plaintext. Recover the payload by undoing the transport encoding first, then the byte transform.

Open these files: 03-envelope.bin.

1. Open 03-envelope.bin and inspect Strings for the transform, key, encoding, and payload.
2. Send the Base64 payload to Decoder lab or paste it there. Choose Base64 to text and Run transform.
3. Copy OUTPUT back into INPUT. Select UTF-8 text XOR to text, enter HEX KEY 2a, and Run transform again. This fixture deliberately decodes to ASCII at the intermediate step.

Read what happened

Find the payload field at 0x100 and copy only its Base64 text, without the label or unrelated bytes. In Decoder lab apply Base64 first. The output is an intermediate representation, not the final answer: the outer wrapping tells you how bytes are represented, while XOR is another transform on those bytes. Copy the output into the input for the next operation rather than repeatedly transforming the original Base64.

For this specific fixture the intermediate bytes are deliberately text-safe. Use the UTF-8 text route and a one-byte hexadecimal XOR key 2a. Verify how the UI represents the key: hex 2a is one byte, not the two ASCII characters 2 and a. This safe text round trip is not a general rule for arbitrary binary ciphertext. For binary that cannot survive text conversion, preserve byte layers in Recovery instead. Record both transforms in order so another learner can reproduce the result.

Hints, one at a time

1. The Base64 payload begins at 0x100. The nearby configuration names both layers.
2. Undo the outer layer first: Base64 decode, then XOR each resulting byte.
3. The single-byte XOR key is 0x2a (decimal 42). Copy the first decoded OUTPUT into INPUT before applying XOR, and set HEX KEY to 2a.

Response format: `SPECTRA{the_recovered_text}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 04 · The gate

The question. This raw x86-64 routine returns EAX = 1 for one input byte and EAX = 0 otherwise. The input byte is supplied in EDI (0..255). Find the accepted byte by reading the code. Submit it as two lowercase hexadecimal digits inside the flag.

Open these files: 04-gate.x64.bin.

1. Open the file in SPECTRA > Disassembly > Raw range / architecture override. Set file offset 0x0, byte length 0x13 (19 decimal / 0x13 hex), base VA 0x0, x86-64, and Little endian. Click Decode range. Do not enter 0x19: that means 25 bytes.
2. EDI holds the input; mov eax, edi copies it into EAX. xor eax, 0x37 transforms it. cmp eax, 0x6d checks the transformed value, and jne jumps to the zero-return path if it differs. Equality reaches mov eax, 1 and ret.
3. Write the success condition: (input XOR 0x37) == 0x6d. Undo the XOR to find the original input byte. Put its two lowercase hex digits inside SPECTRA{...}; submit the input, not the return value 1. Record your reasoning in Notebook. Native execution is not required.

Read what happened

Decode file offset 0, byte length 0x13, base VA 0, x86-64, little endian. The routine copies EDI to EAX, XORs it with 0x37, and compares against 0x6d. The conditional branch selects the failure path when the transformed value does not equal the comparison constant. Read both return blocks so you know what success means here.

To recover an accepted input, undo the XOR: if input XOR key equals target, input equals target XOR key. Use the displayed constants and verify by substituting your candidate into the original operation. The success return value is not the required input. The same distinction matters in password checks, parsers and validation code: a return code describes the result, while an argument is what produces it. A static derivation is enough for this deliberately tiny case.

Hints, one at a time

1. Read from top to bottom. mov eax, edi copies your input. xor changes the copy; cmp compares it with the target. jne means jump if not equal, so success must avoid that jump. The 1 returned on success is a result, not the flag input.
2. XORing twice with the same key restores the original value: (input XOR key) XOR key = input. So input = 0x6d XOR 0x37. XOR works on bits: equal bits give 0, different bits give 1. It is not addition or multiplication.

3. Write 0x6d as 01101101 and 0x37 as 00110111. XOR each column, then convert the two groups of four result bits to hex. Alternatively, reconstruct this function: unaff_EDI is the incoming EDI value; the hex constant in its equality check is the accepted input. Wrap the two lowercase hex digits in SPECTRA{...}, without 0x.

Response format: `SPECTRA{xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 05 · One-byte betrayal

The question. A patch changed two bytes: a revision counter and a branch opcode. Find the executable change and explain how it reverses acceptance. The raw x86-64 routine begins at file offset 0x40 and is 19 bytes long. Submit the four-digit file offset, old byte, and new byte separated by underscores.

Open these files: 05-gate-before.bin, 05-gate-after.bin.

1. Open 05-gate-before.bin as the main file. In Binary diff, use Open second binary to load 05-gate-after.bin, then click Compare. No disassembly setup is needed for the raw byte table.
2. Read Raw byte comparison above Matched functions. Its columns are FILE OFFSET, BEFORE (HEX), AFTER (HEX), and CHANGE. Ignore the revision at 0x10. Record the other changed offset and both byte values. An empty Matched functions table does not block this step.
3. With BEFORE still the main file, go to Disassembly > Raw range / architecture override. Set file offset 0x40, byte length 0x13 (19 decimal / 0x13 hex), base VA 0x40, x86-64, Little endian. Click Decode range. Note the conditional jump after cmp and its destination.
4. Use the main Open file button to load AFTER. Repeat ALL settings from step 3 and click Decode range. Compare its jump with your BEFORE notes: what condition now takes the same branch?
5. Build the flag from the absolute file offset padded to four hex digits, the BEFORE opcode byte, and the AFTER opcode byte, separated by underscores. Use lowercase hex without 0x prefixes.

Read what happened

Set 05-gate-before.bin as the main file and open the after file in Binary diff. Raw byte comparison finds changes without a PE header or recovered function. One difference is revision metadata at 0x10; the behavioral change sits inside the code range beginning at 0x40. Record each offset and classify it before deciding which belongs in the answer.

Decode the routine in each main file separately: offset 0x40, length 0x13, base VA 0x40, x86-64, little endian. Find the jump eight bytes into the routine. Its opcode changes the

condition while its displacement preserves the destination. Explain which input relation now takes that edge. The requested offset is the absolute file offset, not the position within the 19-byte routine. This exercise teaches a patch-review habit: first locate changes, then show which one changes a decision.

Hints, one at a time

1. Binary diff > Compare > Raw byte comparison shows both changes without decoding. Use its FILE OFFSET and BEFORE/AFTER columns; no matched function is required. Disassembly then explains what the code change does.
2. The changed opcode turns jne (jump if not equal) into je (jump if equal). The destination stays the same. Compare the jump row in each file; the first BYTES value is its opcode. The byte after it is the unchanged displacement, not the answer.
3. The instruction begins 8 bytes into a routine starting at file offset 0x40. Pad that absolute file offset to four hex digits, followed by the original opcode byte and replacement opcode byte, separated by underscores inside SPECTRA{...}.

Response format: `SPECTRA{oooo_aa_bb}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

27. Glass Relay: maps, references and bounded records

You now know how to find bytes. The next question is how the program reaches them. This campaign connects PE coordinates, instruction-relative references, bit constraints, a helper call and a malformed record. Keep offsets and VAs labeled on every page of your notes.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/002/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · Map the relay	PE sections + RVA mapping
02 · Follow the reference	RIP-relative data references
03 · Switchboard	Bit masks + TEST
04 · Return path	Calls + value flow
05 · Fractured records	Variable records + bounds checks

Stage 01 · Map the relay

The question. A PE header describes where bytes live on disk and in memory. Find the absolute FILE OFFSET corresponding to RVA 0x2040. An RVA is relative to the image base, not the start of the file. Submit the offset as four lowercase hex digits.

Open these files: 01-relay-map.bin.

1. Open 01-relay-map.bin in SPECTRA. File structure should identify PE32+ / x86-64. Find the section containing RVA 0x2040.
2. Read the section VA and subtract the image base to obtain its RVA. Use: file offset = raw offset + (target RVA - section RVA). This target is file-backed, not zero-filled memory.
3. Verify with File structure > Address mapping ledger: Input space PE RVA, Location 0x2040, Resolve address. Use Inspect original bytes to confirm the GLASS RELAY record. Save your calculation in Notebook.

Read what happened

The section table supplies a coordinate conversion, not a search hint to ignore. Locate the section containing RVA 0x2040. Its RVA start is 0x2000 and raw file start is 0x400. Subtract the section RVA to get displacement within the section, then add that displacement to the raw start. Check that the displacement remains within the file-backed span.

Use File structure's Address mapping ledger with Declared file headers, Input space PE RVA and Location 0x2040. Resolve, inspect the original bytes, and compare with your arithmetic. This independent representation helps catch a decimal/hex or base mistake. Write RVA, file offset and the bytes you inspected in separate columns; the tool cannot fix a report that labels the right number with the wrong coordinate name.

Hints, one at a time

1. The image base is not a file offset. Look at the .rdata section.
2. For .rdata, section RVA = 0x2000 and raw offset = 0x400.
3. The target is 0x40 bytes into .rdata. Add that displacement to its raw-file offset; pad to four hex digits.

Response format: `SPECTRA{rva_oooo}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 02 · Follow the reference

The question. Three channel strings exist, but the entry routine returns the address of only one. Identify the referenced channel value. A string's presence alone does not prove the code uses it. Submit the value after channel=, without the prefix.

Open these files: 02-relay-reference.bin.

1. Open the file and note the three channel= strings. Open Disassembly and choose Analyze code for this PE file.
2. At entry VA 0x140001000, inspect lea rax, [rip + displacement]. Use its outgoing data reference or Cross-references to follow the target; ret returns that address.
3. Verify the target string using its VA/RVA and file offset. If a reference link is unavailable, calculate target RVA = next instruction RVA + signed displacement, then map it to disk as in stage 01.

Read what happened

Several readable channel strings are deliberately plausible. The one the entry routine actually addresses is established by its LEA, not by search order. For RIP-relative addressing, the base is the address of the next instruction. The entry starts at RVA 0x1000 and the LEA occupies seven bytes, so the next instruction is at RVA 0x1007. Add the signed displacement from the operand, then map the target to stored bytes.

LEA calculates an address; it does not read the pointed-to string. The resulting pointer is the value returned by this tiny routine. Follow it to Hex or Strings and verify the complete text. Keep the other strings as decoys in this exercise, not as evidence of several executed

channels. In real code a reference is a relationship; reachability and actual runtime use still need their own evidence.

Hints, one at a time

1. Only the pointer loaded by the entry routine is relevant. The other strings are decoys.
2. RIP-relative addressing uses the address AFTER the lea instruction. It is 7 bytes long.
3. The next-instruction RVA is 0x1007 and the displacement is 0x1079. Follow their sum to the matching string; do not pick the first string.

Response format: `SPECTRA{channel_value}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 03 · Switchboard

The question. The relay accepts exactly one four-bit mode (EDI from 0 through 15). Bits are numbered from zero at the least-significant bit. Read the TEST and AND checks to find the mode that returns EAX = 1. Submit it as two lowercase hex digits.

Open these files: 03-switchboard.x64.bin.

1. Open Disassembly, expand Raw range / architecture override, and set file offset 0, byte length 25, base VA 0, architecture x86-64. Choose Decode range.
2. TEST computes a bitwise AND for flags without changing the operands. A zero result sets ZF: je/jz follows that zero result; jne/jnz follows a nonzero result. Note which branches lead to the zero return.
3. AND changes EAX. Combine the required bits with the low-two-bit equality check. Write the four-bit pattern in your Notebook, then convert it to hex.

Read what happened

Decode offset 0, length 25 decimal, base VA 0, x86-64, little endian, and select All decoded instructions. Write each condition as a requirement on the input bits. TEST computes a bitwise AND for flags without storing the AND result back into the input. A branch based on zero therefore tells you whether the selected bits were clear, not whether the entire input was zero.

The first gate requires bit 2 set; the next requires bit 3 clear. The final low-bit mask isolates bits 0 and 1 and compares their result with 1. Construct a candidate with only the required bits, then substitute it through every gate in order. Keep unrelated upper bits separate from the low-bit question. The useful skill is turning assembly into constraints you can check, rather than guessing byte values until a checker accepts one.

Hints, one at a time

1. The first TEST requires bit 2 to be set; the next requires bit 3 to be clear.
2. The low-two-bit mask is 0x03. Those two bits must equal binary 01.
3. Combine bit 3 = 0, bit 2 = 1, and low bits = 01. Convert that four-bit pattern to one byte.

Response format: `SPECTRA{mode_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 04 · Return path

The question. The entry routine accepts one byte in EDI and passes it in EAX to a local helper. Follow CALL and RET to discover the helper's transformation, then find the input that makes main return 1. Inputs are limited to 0..255; arithmetic here does not overflow.

Open these files: 04-return-path.x64.bin.

1. In Disassembly, expand Raw range / architecture override: file offset 0, length 39, base VA 0x4000, x86-64; Decode range.
2. Follow the CALL from main to the helper at VA 0x4020 (file offset 0x20). LEA EAX,[RAX+RAX*2] computes arithmetic without reading memory; then ADD changes the result. RET returns to the instruction after CALL.
3. Back in main, read the CMP target and solve the helper's expression for the original input. Record the helper expression, accepted byte and failure path. No debugger execution is required.

Read what happened

Decode offset 0, length 39 decimal, base VA 0x4000. Follow the call to helper VA 0x4020, whose file offset is 0x20 under this flat map. Track the value passed into the helper and returned in EAX. LEA with an expression like RAX plus RAX times two performs arithmetic here; there is no dereference of that computed address.

Summarize the helper algebraically as three times the input plus seven, then follow the caller's comparison against 0x7f. Solve backward for an input and substitute forward to check. Record separately the input, intermediate product, helper return and branch outcome. A call hides instructions from the caller's linear listing, but it does not erase the data dependency. This is your first compact interprocedural analysis.

Hints, one at a time

1. Track EAX through the helper before interpreting the comparison in main.
2. The helper computes three times the input, then adds 7. Main compares the result to 0x7f.

3. Convert 0x7f to decimal, subtract 7, divide by 3, then convert the input back to two hex digits.

Response format: `SPECTRA{call_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 05 · Fractured records

The question. One length field claims bytes beyond the file. Header: magic at 0x00 (4 bytes), total file size at 0x04 (u16 LE), record count at 0x06 (u16 LE). Records start at 0x10. Each is tag (u8), payload length (u16 LE), then that many payload bytes; no padding. Find the FIRST invalid record header offset and its claimed end offset (exclusive). Submit both as four lowercase hex digits.

Open these files: 05-fractured-records.bin.

1. Open Hex explorer. Confirm the file size against the header, then start at the first record offset 0x10.
2. For each record: payload start = header offset + 3; claimed end = payload start + length. The next record begins at that end. Check that the full header and payload fit before advancing.
3. Stop when claimed end exceeds the actual file size. Record that record's HEADER offset and its claimed exclusive end. Do not invent missing bytes or trust a length just because it is in the file.

Read what happened

The file is only 0x40 bytes long. Records start at 0x10 and each has a three-byte header, followed by its declared payload. Walk by header size plus payload length; do not search for a plausible next tag in the middle of a payload. Keep a cursor ledger with start, length, exclusive end and remaining bytes.

After the first two records, the third header begins at 0x23. Its little-endian length is declared by bytes 30 00. Calculate its claimed exclusive end and compare that with the actual file length before trying to read the payload. A truncated record remains truncated. Do not fill missing bytes with zeroes or continue from a guessed location. This is the same discipline that prevents a parser from treating untrusted length fields as permission to read beyond a buffer.

Hints, one at a time

1. The actual file size is 0x40 bytes. Valid offsets stop at 0x3f; an exclusive end of 0x40 is valid.
2. The first two payload lengths are 5 and 8. Include each 3-byte header when locating the next record.

3. The third header is at 0x23. Its 30 00 length is 0x30, not decimal 30. Calculate 0x23 + 3 + 0x30 and compare with 0x40.

Response format: `SPECTRA{hhhh_eeee}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

28. Cold Storage: investigate without inventing a malware story

This campaign looks more like a triage case: imports, appended data, encoded configuration and compact routines. The files remain benign teaching artifacts. Your task is to recover what the evidence supports and resist the urge to turn every clue into proof of execution.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/003/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · First look	Import triage + evidence
02 · After the image	Overlay + raw file boundaries
03 · Unwrap the configuration	Repeating XOR + recovered config
04 · Count the bytes	MOVZX + bounded loop + checksum
05 · Words on the stack	Stack strings + write order
06 · Signed or unsigned	Unsigned JBE + byte interpretation
07 · Close the case	Evidence-based analyst report

Stage 01 · First look

The question. Start a static investigation. Which imported DLL owns WinHttpSendRequest? Submit its name in lowercase, including .dll. An import suggests a capability; it does not prove the function was called.

Open these files: 01-cold-storage.bin.

1. Open 01-cold-storage.bin in SPECTRA. In File structure, note PE32+, x86-64, three sections and the imports. Find WinHttpSendRequest and its library.
2. Inspect Disassembly > Analyze code at the PE entry. Record what the entry actually does. Keep that separate from what the import names suggest.
3. Save two Notebook observations: the imported capability and the observed entry instructions. Do not run the sample or browse any extracted URLs.

Read what happened

Inspect the PE headers, sections and imports before drawing a malware story. Identify the imported request-sending API and its owning library exactly. Then inspect the entry code: this fixture's entry simply clears EAX and returns. An import can be present without the entry calling it.

Write two separate statements: the import table declares a symbol; the inspected entry contains no call to it. Do not extend the second statement into a proof about every possible path without examining them. This benign specimen makes the distinction obvious so you can carry it to real malware, where suggestive API names are tempting shortcuts. Detect results and strings are useful leads, but a capability claim needs code and context.

Hints, one at a time

1. Use the parsed import table, not just the string list.
2. Find WinHttpSendRequest under the HTTP-client library. CreateFileW belongs to a different library.
3. DLL spelling is case-insensitive on Windows, but flags are case-sensitive: lowercase the owning DLL name and keep .dll.

Response format: `SPECTRA{library.dll}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 02 · After the image

The question. Find the bytes appended after the PE sections. Compute the largest raw offset + raw size, then inspect that offset. The overlay header is 16 bytes: magic[4], key length u8, transform u8, payload length u16 LE, key bytes at +8, padding. Payload follows at +0x10. Submit overlay offset and payload offset, four lowercase hex digits each.

Open these files: 01-cold-storage.bin.

1. Use File structure section raw ranges. Calculate max(raw offset + raw size); do not use SizeOfImage, which describes mapped memory.
2. In Hex explorer inspect the calculated file offset for CST3. Read key length, payload length and key bytes. Record the payload start and exclusive end; verify the full payload fits the file.
3. This fixture has no certificate table. In real PE files, trailing data may be legitimate; an overlay alone is not proof of malware. Save the offsets for stage 03.

Read what happened

Locate the last stored section's raw extent using file offsets and raw sizes, not its virtual allocation. For this fixture the last .idata section begins at 0x600 with raw size 0x200. The bytes beyond the section's exclusive end include a CST3 overlay header. SizeOfImage is a memory-layout value and is not the correct way to locate this appended data.

Read the overlay header and validate its declared payload range against the actual file. The payload begins after that header, not at the signature itself. This fixture has no certificate directory complication, but real PE overlays can include legitimate data, signatures or packaging. “Bytes after the image's mapped sections” is a structural observation; “malicious hidden payload” would require more evidence.

Hints, one at a time

1. The final raw section is .idata, starting at 0x600 with raw size 0x200.
2. Raw offset + raw size gives the first byte after that section. The overlay starts there, not at the larger SizeOfImage.
3. Add the fixed 0x10-byte overlay header to obtain the payload start. Use four digits for each offset.

Response format: `SPECTRA{overlay_hhhh_pppp}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 03 · Unwrap the configuration

The question. Transform 1 means repeating-key XOR. Use the key length and key bytes from the overlay header; start at key index zero for the payload. Recover the profile value and submit it. The configuration is obfuscated ASCII, not a live destination.

Open these files: 01-cold-storage.bin, 03-payload.hex.txt.

1. Use the payload range from stage 02. For easier copying, 03-payload.hex.txt contains exactly those ciphertext bytes as hex; open it as text and copy its contents.
2. In Decoder lab, choose Hex → UTF-8 text and run the transform. COPY the output back into the INPUT field. This fixture deliberately has printable ciphertext, so this text conversion is lossless.
3. Choose UTF-8 text XOR → text. Enter the header key as hex bytes and run. Read the semicolon-separated fields; save profile, host, marker and mode in Notebook. Repeating XOR wraps the key after its final byte.

Read what happened

Use the provided payload hex text as a reproducible copy of the bounded overlay payload. In Decoder lab perform Hex to text, then apply the repeating XOR key from the header as bytes 01 02 03. The fixture's intermediate values are deliberately printable so this text route is lossless here. Copy the actual output forward between steps and record the transform order.

Read the recovered profile, host, marker and mode as configuration fields. A host string is stored data, not a connection. The key is three binary bytes, not six ASCII digits. A repeated key cycles by byte index; if you begin at the wrong payload offset, the phase can be wrong even with the correct three key values. Check several fields and the declared payload bounds instead of accepting the first readable fragment as validation.

Hints, one at a time

1. The key length is 3. The key begins eight bytes after CST3; the payload begins sixteen bytes after CST3.
2. The key bytes are 01 02 03. They are hexadecimal bytes, not the ASCII characters 0, 1, 0, 2, 0, 3.
3. Hex decoding and XOR are two separate transforms. Replace INPUT with the first output before running XOR, then take only the value after profile=, before the next semicolon.

Response format: `SPECTRA{profile_value}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 04 · Count the bytes

The question. Trace this separate teaching routine with RSI pointing at the six bytes in 04-marker.bin (the config marker). EAX and ECX start at zero inside the routine. It returns a simple sum, not a cryptographic hash. Submit final EAX as four lowercase hex digits.

Open these files: 04-byte-counter.x64.bin, 04-marker.bin.

1. Open the raw code in Disassembly > Raw range / architecture override: file offset 0, byte length 18, base VA 0, x86-64; Decode range. Set FUNCTION to All decoded instructions.
2. MOVZX reads one byte at [RSI+RCX] and zero-extends it into EDX. ADD accumulates it in EAX; INC advances ECX. CMP and JB decide whether to repeat.
3. Read the six marker bytes in Hex explorer. Make a Notebook table: index, byte value, running sum. Stop at the loop bound and convert the final sum to four hex digits.

Read what happened

Decode the small checksum routine with file offset 0, length 18 decimal, base VA 0. MOVZX loads a byte through RSI plus the current index and zero-extends it before accumulation. Track the accumulator and index as separate columns. The comparison and unsigned JB define how many bytes are processed.

Use the six marker bytes given in the recovered configuration, 52 41 56 45 4e 37, exactly once each. Add numeric byte values, not the digits printed in the hex display and not the string's character count. Check whether the terminator is included by following the loop bound: do not assume every string routine consumes its trailing zero. The final accumulator is a property of this defined byte range and algorithm, not a cryptographic identity for the file.

Hints, one at a time

1. RCX is the byte index. The comparison is against 6, and JB loops only while the index is below that bound.
2. The six byte values are 52 41 56 45 4e 37 in hex. MOVZX means values 0..255, not signed bytes.
3. Add 0x52 + 0x41 + 0x56 + 0x45 + 0x4e + 0x37 once each. Check six iterations and convert the sum, not the text, to hex.

Response format: `SPECTRA{sum_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 05 · Words on the stack

The question. This separate routine builds a null-terminated string on its stack using immediate writes. Recover the string at RSP before the stack allocation is released. Submit the lowercase text, without the null byte. The writes are intentionally out of address order.

Open these files: 05-stack-note.x64.bin.

1. Decode the raw x86-64 file at offset 0, length 32, base VA 0x5000. Set FUNCTION to All decoded instructions. SUB RSP,0x10 reserves sixteen bytes; ADD RSP,0x10 later releases them.
2. For each MOV dword ptr [rsp+offset], immediate, expand the immediate into four little-endian bytes. Put those bytes at their destination offsets in a sixteen-byte Notebook grid.
3. Read increasing addresses from offset zero until the first 00 byte. Do not concatenate values in instruction order. No native execution is needed.

Read what happened

Decode offset 0, length 32 decimal, base VA 0x5000. The routine allocates stack space and writes immediate DWORD values at different offsets. Instruction order alone is not the string's character order. Build a byte map indexed by destination offset, splitting each DWORD into little-endian bytes at its own destination.

The writes cover stack offsets +4, +8 and +0. Reassemble by address, then read only the initialized text through its terminator. Bytes 0 through 10 form the text and byte 11 terminates it; unused allocated stack bytes are not automatically part of the message. This is a stack string: text assembled through stores rather than present as one contiguous literal in the file. Static store reasoning can recover it without a native process.

Hints, one at a time

1. Writes target RSP+4, RSP+8 and then RSP+0. Memory order differs from instruction order.
2. For each 32-bit immediate, the least significant byte occupies the lowest address. For example 0x44434241 stores ASCII ABCD.
3. After arranging the writes by destination, read offsets 0 through 10. Offset 11 terminates the string; uninitialized bytes after it are irrelevant.

Response format: `SPECTRA{stack_string}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 06 · Signed or unsigned

The question. Supply the recovered config mode (0x80) in DIL to this separate teaching routine. Determine the final EAX return code. DIL is the low byte of RDI. Submit the return code as two lowercase hex digits.

Open these files: 06-route-byte.x64.bin.

1. Decode the raw x86-64 file at offset 0, length 20, base VA 0. Set FUNCTION to All decoded instructions. Follow MOVZX EAX,DIL and CMP AL,0x7f.
2. JBE means unsigned below-or-equal (CF=1 or ZF=1). Decide whether the branch is taken, then follow that path to its MOV EAX and RET.
3. Write down both possible interpretations of 0x80: unsigned 128 or signed -128. This routine uses zero extension and an unsigned condition. Explain why the signed interpretation gives the wrong path.

Read what happened

Decode offset 0, length 20 decimal, base VA 0. MOVZX interprets the low input byte as an unsigned value. CMP AL,0x7f sets flags, and JBE means unsigned below or equal. For the supplied 0x80 input, compare unsigned 128 against 127; interpreting the same bits as signed negative 128 would answer a different branch question.

Follow only the selected edge to its returned immediate. Do not keep walking through a RET into the next displayed block. Record the operand width, comparison, flags condition and chosen return. When a decompiler prints a signed type, this is exactly the kind of branch to audit against the original condition code.

Hints, one at a time

1. The compare operates on AL, one byte. MOVZX copies DIL without sign extension.
2. Unsigned 0x80 is larger than unsigned 0x7f. JBE is different from signed JLE.
3. When JBE is not taken, execution falls through to the next MOV EAX. Use that immediate as the return code, padded to two hex digits.

Response format: `SPECTRA{route_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 07 · Close the case

The question. Choose the draft conclusion in 07-report-candidates.txt that matches the evidence. Submit its lowercase letter plus the hostname recovered in stage 03. Then, optionally, complete the report template or export your Notebook. A good analyst distinguishes facts, hypotheses and unknowns.

Open these files: 07-report-candidates.txt, ANALYST-REPORT.txt.

1. Review the PE entry and imports, overlay offsets and decoded config. The separate raw snippets teach patterns; they are not established reachable functions in this PE.
2. Compare the three report drafts. Reject claims of actual network contact, persistence or a named malware family that this static exercise did not demonstrate.
3. Combine the supported draft letter and the recovered host field with an underscore. Use the host alone: no protocol, path or semicolon. Keep your report private so others can investigate.

Read what happened

Read the report candidates after assembling your own evidence note. Include the declared import, bounded overlay, recovered configuration, loop result and stack-string method. The separate snippets are teaching artifacts; do not invent a call graph connecting them to the PE entry merely because the campaign groups them together.

Choose the report whose claims can be traced to those observations. Reject claims of live network activity, persistence or a malware family if the supplied evidence does not demonstrate them. The response format combines a report choice with the recovered host, without adding a URL scheme that was not present. Export your notes and sample identities. The important finish is a reproducible explanation, not a dramatic label.

Hints, one at a time

1. An imported function or embedded URL is a lead, not a behavior log. A training label is not malware-family attribution.
2. The PE entry returns zero without calling imports. These exercises never demonstrate execution or live network traffic.
3. Select the draft that explicitly separates static evidence from unsupported claims. Pair its lowercase letter with the host= value from your decoded configuration, not the visible decoy URL.

Response format: `SPECTRA{letter_hostname}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

29. From first cases to connected investigations

Finishing Cold Storage means you can connect bytes, locations and small routines without inventing behavior. The next five campaigns build on that habit. You do not need to recognize every instruction on sight. You do need to explain which bytes support a claim and what would make you revise it.

Next campaign	New responsibility
004 / Ghost Circuit	Carry arguments and record fields through calls. Resolve a selected indirect target from its actual table.
005 / Hollow Signal	Track a rolling transform and parser bounds; ask whether a check happens before the read it protects.
006 / Iron Veil	Keep hash collisions as candidates; distinguish a status from a result and identify missing test coverage.
007 / Black Loom	Reconstruct a packed image, preserve coordinate spaces and establish what a failed reconstruction actually produced.
008 / Puppet Core	Recover a small interpreter's instruction meanings, virtual state and branches; compare a changed virtual program.

A readiness check you can do without guessing a flag

1. Open a raw routine from 001 and explain file offset, byte length and base VA. Enter 0x13 for nineteen bytes; 0x19 means twenty-five.
2. Read one conditional jump. Write the tested condition, taken destination and fall-through destination. A destination address is not the address of the changed opcode.
3. Compute one eight-bit operation and explicitly keep only the low byte. Explain why a 32-bit operation could differ.
4. Write a three-line note: observed bytes, your interpretation, and one uncertainty. Preserve the exact file hash.

If one item is uncomfortable, revisit that lesson before moving on. Difficulty is not a race. The intermediate book supplies a complete chapter for each campaign from 004 through 008, with steps, explanations, hints and checks. Use the chapters in order and return to the reference chapters as needed.

What changes after the first three campaigns?

A single attractive string no longer settles a question. Later cases ask you to maintain a small model across several instructions, distinguish equivalent-looking addresses, and test alternatives. The task is still bounded: the supplied maps and format notes define what is known. You are not expected to guess undocumented constants or run a sample on your machine.

Use `/spectra` in the Church's challenge-crypt channel to continue privately. Start with 001; the menu keeps your place. Website casebooks remain available without Discord. Completion requires the campaign's actual flag checks in Discord; reading this book does not change your progress.

30. From packed bytes to interpreted behavior

A recovered file can contain instructions for another program rather than instructions for your CPU. This is the bridge between Black Loom and Puppet Core. First ask how bytes are stored, then ask who interprets them.

Keep three representations separate

Representation	The question it answers
Container	Where does the meaningful byte window begin and end on disk?
Recovered image or bytecode	What did reconstruction produce, and which offsets belong to that output?
Native interpreter	Which CPU instructions read and implement the custom commands?

In 007 you reconstruct an image, then inspect its code at the supplied base. In 008 the container supplies a virtual program, while interpreter.bin is the native x86-64 code that gives that program meaning. Disassembling every container byte as x86-64 may produce plausible text. That text is not proof those bytes are CPU instructions.

A small mapping exercise

For an unrelated toy container, suppose bytecode starts at file offset 0x30 and its virtual program counter is 0x06. The absolute file offset is 0x36. If those bytes are copied to memory base 0x9000, their virtual-program memory address is 0x9006. Neither number tells you the native interpreter's current instruction address. Write the coordinate system beside every number.

Trace a toy instruction language

Suppose a toy instruction ADD 03 changes an eight-bit accumulator A from fe to 01, then advances virtual PC by two. That is a virtual instruction implemented by several native CPU instructions. A trace row should record the PC before the instruction, its bytes, A before and after, and the next PC. Do not count the native instruction addresses as virtual byte offsets.

Now suppose a virtual compare saves its equality result in a separate variable Z. A native bounds check might change the CPU's own zero flag afterward. The interpreter can still branch using saved Z. In 008 you will locate the register holding that saved virtual flag rather than assuming the most recent native CMP controls every virtual branch.

Know what a failed run means

Use a pair of observations: status and result. A successful run returning zero can mean rejection. A truncated input, unknown instruction or exhausted analysis budget is a different outcome; its result may be unavailable. Record that distinction before discussing acceptance. An analysis error is not evidence that a program rejected an input normally.

Carry this into future research

This habit helps when examining script engines, configuration interpreters and virtualized code. Recover the grammar and state transitions, preserve bounds and provenance, and keep unknown behavior explicit. The teaching interpreter is deliberately small. Its checks do not make it a general-purpose hardened sandbox, and its eight commands do not describe every real interpreter.

Continue with the intermediate chapters for 007 and 008 only after the earlier campaigns. The 008 trace worksheet is a checking aid: predict a few rows on paper first, then compare. If the model disagrees with your notes, inspect the first divergent row rather than trying unrelated flags.

31. Every section: the desk reference

Use this chapter when you know the question but cannot remember which panel answers it. The case chapters teach the workflow; this index explains smaller controls and the traps around them. The existing Training ground field guide remains the detailed control reference shipped with SPECTRA.

Strings

Search filters extracted text; it is not a general regex engine. Minimum length controls how short a candidate can be. Lowering it finds more small fragments and more noise. Encoding selection changes which representations are shown. Unique view can hide repeated occurrences; turn it off when every offset matters. Tags such as URL or path are heuristic classifications.

Select a row for its inspector. Copy, bookmark, View bytes, Find code references, and Decode each preserve different parts of the workflow. A preview can be clipped; inspect the full relevant bytes before relying on a long value. Pagination/filtering affects what you see, not necessarily every result in the underlying pass.

Disassembly

Analyze code builds a bounded recursive pass from available seeds. The function selector narrows context; the filter searches displayed code; Jump to VA uses an address in the mapped image. Assembly and Blocks offer linear and graph-oriented views. **Show reconstructed C beside assembly** supplies context without making the reconstruction authoritative.

Instruction context exposes addresses, bytes, annotations, references, and actions such as **Debug from here**. A mid-function entry may need state established earlier. **Continue analysis** increases a partial pass in bounded increments; the initial budget is 60,000 instructions/4 MiB decoded, with extensions up to 1,000,000/16 MiB. Read deadline and partial-result warnings. More decoded bytes do not automatically resolve every indirect edge.

Raw range / architecture override supplies file offset, length, base VA, CPU, and byte order. It replaces current code results with the stated interpretation. Use it for a known code range, not to force an unsupported whole executable to “work.” Export code JSON when you need evidence beyond a visible table.

Functions and Cross-references

Functions lists candidate starts, names, instruction counts, known callers, and boundary evidence. A count is bounded by recovery. Library-function evidence distinguishes declared imports, forwarding thunks, and exact-byte candidates. **Use this name** records an analyst choice. Signature pack import/export works with reviewed byte patterns; no match means unknown.

Cross-references records source, kind, destination, and evidence. Calls, branches, imports, string/address references, indirect transfers, and switch cases have different meanings. Context filters can restrict the view to one string or function; **Show all references** clears that context. An analyst-added destination remains a hypothesis, not runtime confirmation.

Decompile

Choose native Ghidra or managed ILSpy. Use selected function, Find functions, or a justified custom VA. Native controls include architecture, byte order, compiler/ABI, raw mapping, applied signatures, and one type declaration. Structure workspace names fields, previews file bytes, and supplies saved declarations. Packed/nested/bitfield layouts may need manual declarations.

The managed tree supports search, category filters, method/type selection, and metadata dependencies. Harness lab selects methods, a first method, SDK/CPU target, and optional setup recipe before exporting source and clues. It is an external runner generator. **Copy code**, **Export source**, and **Export evidence** serve different documentation needs. Inspect warnings and refresh reconstruction after corrections.

Debugger lab

Setup selects environment, CPU, byte order, start, raw/load assumptions, and a function harness when appropriate. Execution controls step/run/pause/reset. Breakpoints inspect instruction locations; watchpoints inspect memory access. Register edits and session patches are deliberate changes. Memory map explains which addresses exist and their permissions.

Experiment setup supplies arguments and buffers. Environment recipes supply explicit mapped memory, virtual files, API hooks, stdin/argv/env, or supported ARM MMIO. Inputs apply on Initialize. Batches run fresh cases; interactive edits and breakpoints are not inherited. Saved experiments, reports, coverage, and traces have bounds and need their stop reasons preserved.

Hex explorer and File structure

Hex navigation uses original file offsets. Interpret selected bytes with size and endianness in mind. The original view is read-only. File structure explains format fields, regions, dependencies, and available metadata. Use it before choosing a CPU/loader. A malformed or unsupported field can limit the parser; do not treat a sparse panel as an exhaustive absence claim.

Detect

Scan with DiE, Heuristics, Deep scan, raw report, and report export form a separate detection pass. The 16 MiB file cap is smaller than the main workbench's 64 MiB cap. First-use engine downloads and the 180-second deadline can affect a scan. A canceled or timed-out scan is not a clean verdict.

Recovery and Decoder

Decoder is for quick text transformations: Base64, hex-to-text, URL decoding, and text XOR with a hex key. Output can be invalid UTF-8 if the underlying data is binary. Copy important results and document their origin.

Recovery takes an original/layer input, offset, and length. Hunt candidates, inspect stack writes, or apply a justified known transform. Candidate details show method/source evidence. Decoded layers can become another input, be downloaded, or be saved in a full project. **Clear results** removes recovery work; export first. Transform limits are 4 MiB input, 8 MiB output, eight layers, and 16 MiB total layer data.

Binary diff and Firmware files

Diff holds a second local binary, compares bounded evidence, exposes matches/ambiguity, and can reconstruct selected native pairs. It is not a semantic proof of patch correctness. Preserve both hashes and match basis.

Firmware inspection starts at a specified container offset. Entry metadata, byte downloads, child investigations, and inventory exports are separate outputs. Save the parent before opening a child. Inventory is not a verified software bill of materials or vulnerability verdict.

Training ground and Notebook

Training ground offers explanations, exercises, evidence checks, lesson sources, offline/source downloads, and community links. A passed exercise checks selected expected evidence, not universal mastery. The source download lets you inspect how a fixture was made.

Notebook has address-based code notes, names, bookmarks, signature hypotheses, saved evidence, local investigation revisions, workspace save/restore, and portable projects. Use signature-to-Ghidra controls deliberately. Local-variable notes remain notes. The library's Trash can be recoverable; permanent deletion is a different operation. Export before tidying a valuable case.

Android, recordings, mapping and saved work

Android: Read APK / DEX builds an inventory; Decompile reconstructs a selected class; Inspect method bytes returns original DEX evidence. Manifest/resources decoding is separate from source reconstruction. Source references are symbolic relationships, not exact source-line instruction maps.

Persistent recording: start before execution, stop and flush before export, then save the completed file. Read-only imported observations never initialize an emulator. **Routine history:** begin immediately after initialization in supported x86/x64 Bare routine mode; use Step backward and Return to latest; preserve useful states separately because the timeline is session-local.

Address mapping ledger: choose declared headers, captured debugger mappings or the current raw range, then label the input as file offset, VA or PE RVA. Zero-fill, allocated memory, file-only bytes and overlaps are meaningful outcomes.

Notebook: prepare exact selected evidence, inspect inventory before download and re-prepare after edits. The investigation library keeps local revisions; portable projects and evidence bundles are your handoff copies. **Training ground:** Check my evidence explains missing observations, and Save lesson workspace preserves a resumable route.

32. When the tool disagrees with you

Stop messages are information. The fastest way out of a confusing state is to identify which layer failed: file identification, analysis, engine loading, mapping, execution, or saving. Changing five settings at once makes the cause harder to learn.

What you see	First useful checks
No strings	Minimum length, encoding, current filter, compressed/encrypted content, actual file type
No code reference	Context/kind filters, incomplete code pass, indirect addressing, mapping, runtime-built data
Nonsense assembly	CPU, Thumb/ARM mode, endianness, code-versus-data range, alignment
Partial analysis	Budget/deadline status; continue a justified pass or narrow the investigation
Ghidra output looks wrong	Function start/boundary, ABI, types, indirect-call evidence, original instructions
ILSpy worker cannot start	Use the hosted page or complete served offline release; retry after checking asset availability and tab resources
Debugger refuses the file	Format/CPU pair, interpreter, imports, relocations, TLS, explicit preflight reason
Unmapped memory access	Pointer versus value, caller-established state, input length, loaded addresses and map
Unsupported API/syscall	Record call site/arguments; decide if static analysis or an explicit limited model can answer the question
Wrong decoded text	Input representation, exact range, encoding, transform, key phase, wrapper, IV/tag/AAD
Save is missing	Correct hash/profile/browser, saved status, storage quota, cleared site data, portable backup

Do not “fix” evidence by hiding it

Blindly skipping a faulting instruction can change the program's meaning. Returning success from every missing API can open paths the real system would reject. Renaming an uncertain function to a familiar library name can make later analysis feel settled without adding proof. Keep assumptions explicit and rerun only the experiment they justify.

Ask for help with a reproducible question

Share the app version, sample hash, platform/format, exact section and controls used, a short error message, and your expected versus observed result. Include a benign reproduction when possible. A screenshot alone may omit the configuration that caused the problem. Do not publish private binaries, credentials, or live samples merely because a support conversation asks what happened.

Example: “On v0.18, the ARM addition lesson returns 5, but my own ARM ELF fails preflight with PT_INTERP unsupported. Is its dynamic loader the missing requirement?” This gives someone enough information to answer usefully.

Protect your attention

When stuck, write the last reliable fact and the next smallest test. “These eight bytes decode correctly in ARM mode; the pointer argument is unmapped” is progress. It narrows the problem. A break, a saved case, and a fresh look often beat another hour of random settings.

Newer workflows: find the failing layer

What you see	What to check next
APK reads but a class fails	DEX version, missing/ambiguous dependencies, conversion warnings and per-operation bounds. Try another small class; do not interpret a failure as source code.
History refuses to start	Fresh initialization, x86/x64 Bare routine, memory and action bounds. Process or other-CPU support elsewhere does not imply history support.
A recording is interrupted	Saved segment status and footer/completion; preserve the known span and mark the missing tail.
Bundle lacks a new note	Prepared inventory is a snapshot. Prepare again after edits; inspect members before downloading.
Lesson check stays incomplete	Correct fixture hash/version and an actual requested observation. Imported text is not a new capture.
CTF answer is rejected	Exact format, decimal/hex, padding, offset versus VA and field versus calculated return. Revisit the bytes before guessing.

33. Finish an investigation you can defend

Choose the Windows model demo, Linux echo, ARM addition, managed LabClient, or synthetic firmware archive. Work without following its chapter line by line. Your goal is a short evidence-backed report, not a huge pile of screenshots.

The assignment

1. Identify the exact artifact and write a narrow question.
2. Explain its format, CPU or managed runtime, and appropriate analysis route.
3. Locate one meaningful string/data value or instruction sequence.
4. Follow the relevant relationship into code. Explain a caller, condition, or data transformation.
5. Make a prediction. If a compatible browser experiment exists, test it and preserve inputs, stop reason, and output. Otherwise state why the result remains static.
6. Save a project plus the narrow report needed to explain your finding.
7. Write an alternative explanation and the next observation that would distinguish it.

A model answer, in miniature

“I investigated the benign Linux echo ELF. Its header identifies x64 little-endian ELF, and its entry prepares a read into scratch followed by a write of the returned count. Under SPECTRA's Linux model with stdin hello plus newline, stdout contains the same six bytes and the process exits zero. With science plus newline, stdout changes to those eight bytes. This supports an echo behavior for these supplied inputs. It does not test arbitrary Linux compatibility or networking.” Add the real fixture hash and captured report identifiers.

Mark your own work

Give yourself one point each for exact identity, correct address spaces, evidence-backed code explanation, explicit assumptions, a justified conclusion, a saved artifact, and a useful unresolved question. Seven points means a reproducible small investigation. Missing points identify the habit to practice; they do not determine whether you are “a real hacker.”

Before moving to the intermediate manual, you should be comfortable distinguishing an offset from a VA, following a string into a caller, reading a simple condition/loop, supplying two integer arguments, interpreting a stop reason, and saving a case. You do not need to memorize the glossary. You need to know when a word signals something you should check.

Bring one saved case into the intermediate manual. The same evidence habits carry into deeper experiments.

34. The vocabulary, without the gatekeeping

ABI

Rules for how compiled pieces cooperate: argument placement, stack alignment, returns, and more. A CPU match alone is not an ABI match.

Address / offset / RVA

An address names a memory location. A file offset names a byte position in a file. A PE RVA is relative to the image base. Always label which one you mean.

API / syscall

An API is an interface offered by software. A syscall is a request into operating-system services. SPECTRA models selected behavior rather than supplying every implementation.

Artifact / provenance

An artifact is the thing being examined. Provenance records where it came from and how it was transformed, including hashes and parent-child relationships.

Basic block / control flow

A block is a straight run of instructions with boundaries at transfers. Control flow is the set of possible paths between them.

Breakpoint / watchpoint

A breakpoint watches execution at an address. A watchpoint watches a memory access. Their pause timing can differ.

Compiler / linker / loader

A compiler translates source into lower-level code. A linker combines code and resolves relationships. A loader maps an executable and prepares entry state.

Decompiler / disassembler

A disassembler decodes instructions from bytes. A decompiler builds a higher-level reconstruction using instructions or managed IL and metadata.

Dependency

Code, data, a runtime, or an external service required by the target. Not every dependency appears in one imports list.

Endianness

The byte order of multi-byte values. It must match the target representation.

Entropy

A statistical measure of byte distribution. It can guide region selection, but it is not a malware score or proof of encryption.

Entry point

The location where a loader begins execution. Startup work may run before the source-level main function.

Hash

A fingerprint of bytes. It identifies an artifact; it does not explain behavior or prove trust.

Heap / stack / BSS

Different memory roles: allocations, call/temporary storage, and zero-initialized data. Some loaded bytes have no stored file counterpart.

Heuristic

A useful rule of thumb that can be wrong. Preserve its evidence and confidence instead of treating the label as proof.

Hook

In this lab, an analyst-supplied replacement for one modeled call boundary. It changes the experiment and must be documented.

Import / export

A requested external symbol versus a symbol exposed for other code to use.

Indirect call

A call whose destination is obtained through a register or memory value rather than fully stated as a fixed target.

Instruction set / ISA

The operations and encodings a CPU family understands. Extensions and modes still matter.

IOC

Indicator of compromise: a contextual clue useful in detecting or investigating malicious activity. A raw string is not automatically an IOC.

Managed / native

Managed code relies on a managed runtime's execution model and metadata. Native code targets machine instructions. Files can contain both.

MMIO

Memory-mapped input/output: accesses to addresses representing hardware registers. A recorded response is not a full device simulation.

Obfuscation / packing

Transformations that make understanding harder, or wrap code/data for later recovery. Legitimate software can use them too.

Pointer

A value interpreted as an address. The pointer and the bytes it points to are different things.

Relocation / load bias

An adjustment to address-dependent data; a load bias is the displacement between linked and loaded locations.

Static / dynamic

Studying stored content versus observing execution. An emulated run is dynamic within its stated model.

Symbol / stripped

A named code/data identity; stripped files have had some helpful metadata removed.

Thunk

A small forwarding or adaptation routine. Follow it to understand the target and any transformation.

Trace / coverage

A record of execution attempts versus a summary of observed locations/transitions. Neither automatically describes all possible paths.

Virtual file

Bytes with a path inside the model. It is not a file written to that path on your real computer.

Xref

Cross-reference: an evidenced relationship between locations. A static xref is not itself a runtime event.

APK / DEX

An Android package and a format holding managed class definitions and bytecode. An APK can also contain resources and native libraries.

Manifest

Structured declarations about an Android app, including components and permissions. A declaration is not an observed action.

TLV

Type, length, value. A record style in which a tag identifies a kind and a length bounds its value; exact widths and length meaning come from the format.

Exclusive end

The first byte after a range. A range starting at 0x20 with length 4 covers 0x20 through 0x23 and ends exclusively at 0x24.

Collision

Different inputs producing the same hash or identifier. Matching a short hash gives candidates, not necessarily unique identity.

Short circuit

An earlier decision ends a path before later checks run. The existence of a later check does not mean every input reaches it.

Modular arithmetic

Arithmetic retaining a fixed width, such as an 8-bit addition keeping only the low eight bits. Carry beyond that width is discarded unless separately used.

Coverage gap

A behavior or class absent from a particular set of observations. Absence from those runs does not prove it is unreachable.

Persistent recording

Ordered observations saved in bounded local segments. It is not automatically a replayable execution state.

Routine history

Supported reconstruction of earlier action boundaries in a bounded x86/x64 bare-routine experiment.

Provenance

Where evidence came from and what transformations, assumptions and identities connect it to the claim.

35. Sources and laboratory notes

Edition 3, 11 November 2026. Written for SPECTRA 0.18 and public CTF campaigns 001-008. Read these manuals alongside the Training ground exercises or keep the PDFs and practice files for offline use. The original app field guide remains a separate control reference.

What was checked

Control names and support boundaries were checked against the current local SPECTRA source and its Android, recording, evidence and routine-history documentation. Campaign files, maps, steps and hints were checked against the eight Church casebooks; fixture hashes and the reference response set were checked during this revision. The first edition's fourteen benign emulator checks remain historical verification, not newly repeated tests. The previous edition passed 22 independent benign-fixture checks, forty CTF reference-response checks, and practice-file hash and document-link checks. Exact practice-file hashes are in practice/manifest.json.

This verification is not a claim that every operating system, browser/device, malware sample, or external harness works. No live malware was executed to write these manuals. Managed example expectations are derived from the included benign source and metadata; external harness execution is not part of the browser lessons.

Practice artifacts

Built-in lesson buttons are the easiest route. The companion folder also supplies exact benign PE/ELF/raw/managed specimens and data. The add-one/add-two files are routine-analysis fixtures. The firmware archive is not bootable. The record and encoded-message files are data. Original SPECTRA fixture licensing is retained in practice/LICENSE.txt. No malware samples are bundled.

Expected results belong to these particular fixtures and stated settings. If your observation differs, save it and inspect the discrepancy; do not edit evidence to match the answer key. Native builds produced from source on another toolchain may have different addresses or instructions.

Primary references and further reading

Primary references from the first edition were retained; Android format and engine references were revisited in November 2026. Formal references are useful for definitions and ABI details; the SPECTRA source establishes what this integration actually implements. The worked explanations and exercises are original teaching material based on the supplied fixtures.

1. SPECTRA source, pinned release commit. Release baseline; revision checks also use the current local source and its feature documentation.
2. SPECTRA CTF collection. Church-authored public casebooks and benign fixture maps for campaigns 001-008; local published assets checked 11 November 2026.

3. SPECTRA Android workflow. Integration controls, limits and fixture interpretation.
4. SPECTRA routine history. Supported replay contract and persistence boundaries.
5. SPECTRA recordings. Segmented observation formats and adapters.
6. SPECTRA evidence workflows. Selective bundles, mappings and lesson evidence; code resolves older documentation conflicts.
7. AOSP DEX format. Managed bytecode and metadata layout.
8. Android manifest overview. Declared components and permissions.
9. dex2jar. DEX-to-class conversion upstream.
10. Vineflower. Java reconstruction upstream; browser bridge capabilities are narrower.
11. Church of Malware Git. Community source and research projects.
12. MalwareBazaar browse. Research sample lookup and current search syntax; not required for the benign exercises.
13. MalwareBazaar FAQ. Current service and sample-handling information.
14. Microsoft PE format. Formal PE field, address, and machine terminology.
15. ELF header specification. ELF class, encoding, and identity fields.
16. ELF program loading. Program-header and loading relationships.
17. Microsoft x64 calling convention. Formal Windows x64 argument and stack conventions.
18. Arm AAPCS32. Formal 32-bit Arm calling rules.
19. Arm AAPCS64. Formal AArch64 calling rules.
20. AMD64 psABI. System V AMD64 ABI project.
21. RISC-V psABI. RISC-V ABI distinctions and formal definitions.
22. Ghidra upstream. Native RE engine project; upstream capability is not a promise about the browser bridge.
23. ILSpy upstream. Managed decompiler project.
24. Detect It Easy upstream. File/compiler/packer identification project.
25. Web Crypto decryption reference. Browser decryption API parameters and errors; SPECTRA exposes a narrower transform UI.

Edition 3 additions

This revision adds the eight stages of Black Loom and nine stages of Puppet Core, bringing the collection to eight campaigns and 57 stages. Beginner bridge chapters explain readiness, coordinate spaces and native versus virtual state. Intermediate chapters include the complete 007 and 008 steps, explanations, hints and reasoning checks. Earlier test totals above describe the previous edition; the new campaign checks are recorded separately.

CTF 008 was checked against an independent model and bounded native CPU emulation for every possible byte input in both program versions, plus malformed-input and state-isolation cases. Its browser trace model was checked against 361 native CPU cases. These checks concern the supplied synthetic fixture, not arbitrary malware. Exact practice bytes are verified against the source campaign files.

SPECTRA flags

Reference solutions for the exact companion campaign fixtures. Keep the derivation with your evidence; an accepted string alone does not explain a binary.

SPECTRA // THE DEAD CHANNEL

001/01 · Carrier signal

`SPECTRA{static_before_dynamic}`

The ASCII flag starts at file offset 0x140 and is NUL-terminated. Save the string plus offset as evidence. Nothing was executed.

001/02 · Wrong end of the wire

`SPECTRA{read_the_other_end}`

The u32 LE pointer is 0x180, the u16 length is 54 bytes, and tag 2 selects UTF-16BE. Decode that slice. ASCII-only scanning misses alternating zero bytes.

001/03 · Wrapped transmission

`SPECTRA{layers_are_not_encryption}`

Decode the Base64 string at 0x100 into bytes, then XOR each byte with 0x2a. The resulting ASCII is the flag. Base64 is the outer representation, not encryption.

001/04 · The gate

`SPECTRA{5a}`

mov eax,edi; xor eax,0x37; cmp eax,0x6d; jne failure. Equality requires EDI = 0x6d XOR 0x37 = 0x5a. The success path returns 1; failure zeroes EAX.

001/05 · One-byte betrayal

`SPECTRA{0048_75_74}`

Offsets 0x10 and 0x48 change. The former is a revision byte. At 0x48, 0x75 (jne) becomes 0x74 (je). Equality now jumps to failure, so 0x5a is rejected and other byte inputs pass. Both jumps target file offset 0x50.

SPECTRA // GLASS RELAY

002/01 · Map the relay

`SPECTRA{rva_0440}`

RVA 0x2040 belongs to .rdata. Raw offset 0x400 + (0x2040 - 0x2000) = 0x440. VA would be image base 0x140000000 + RVA = 0x140002040.

002/02 · Follow the reference

`SPECTRA{glass_signal}`

The entry LEA uses RIP after the seven-byte instruction: RVA 0x1007 + 0x1079 = 0x2080. That maps to file offset 0x480, whose string is channel=glass_signal. The other strings are unreferenced decoys.

002/03 · Switchboard

`SPECTRA{mode_05}`

Bit 2 must be on, bit 3 off, and the low two bits must equal 01. The unique four-bit pattern is 0101 = 0x05. TEST preserves EAX; AND keeps only its lowest two bits.

002/04 · Return path

`SPECTRA{call_28}`

The helper at VA 0x4020 computes 3*x+7 and returns to main at 0x4007. Main requires 0x7f (127), so x=(127-7)/3=40=0x28. Inequality reaches xor eax,eax; ret.

002/05 · Fractured records

`SPECTRA{0023_0056}`

Record headers are 0x10, 0x18, 0x23. The third payload starts at 0x26 with declared length 0x30, claiming an exclusive end of 0x56. Only 0x40 bytes exist, so the third record is invalid; 0x16 claimed bytes are missing.

SPECTRA // COLD STORAGE

003/01 · First look

`SPECTRA{winhttp.dll}`

WINHTTP.dll owns WinHttpSendRequest (and WinHttpOpen / WinHttpConnect). The PE entry is xor eax,eax; ret. Imports suggest an HTTP-client capability but this entry does not call them; presence is not observed traffic.

003/02 · After the image

`SPECTRA{overlay_0800_0810}`

The largest section raw end is .idata: 0x600 + 0x200 = 0x800. CST3 begins there. The 16-byte header puts payload at 0x810; its u16 length is 0x65, ending exactly at file size 0x875. SizeOfImage is not a disk offset.

003/03 · Unwrap the configuration

`SPECTRA{nightjar}`

Repeating XOR with 01 02 03 yields profile=nightjar. The host is telemetry.example.invalid, marker RAVEN7 and mode 0x80. This is reversible obfuscation, not strong encryption. No connection was observed or requested.

003/04 · Count the bytes

`SPECTRA{sum_01b3}`

The loop processes indexes 0..5. Zero-extended byte values 82, 65, 86, 69, 78, 55 sum to 435 = 0x01b3. After INC makes ECX=6, JB is not taken. Exactly six bytes are read.

003/05 · Words on the stack

`SPECTRA{coldarchive}`

At offsets +0, +4, +8 the bytes spell cold, arch, ive followed by 00. Memory therefore contains coldarchive. Writes were deliberately ordered +4, +8, +0; instruction order is not string order.

003/06 · Signed or unsigned

`SPECTRA{route_02}`

MOVZX yields 0x80. CMP AL,0x7f compares unsigned 128 with 127; CF=0 and ZF=0, so JBE is not taken. The fall-through path returns EAX=2. Signed JLE would answer a different question.

003/07 · Close the case

`SPECTRA{b_telemetry.example.invalid}`

Draft B is supported. telemetry.example.invalid is the recovered config host; the visible updates URL is a decoy. Imports and data alone do not prove contact, persistence or attribution. The PE entry only returns zero, and separate snippets are exercises. Your report should state those limits.